Making Deep Networks Trainable

Vanishing gradients, initialization, normalization, and shortcuts

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Why gradients vanish or explode

With identity activations, $\hat{y} = W_L ... W_1 x$

V VISUAL

To isolate depth, set the activation to the identity and every bias to zero.

$$h_1 = W_1 x, \quad h_2 = W_2 W_1 x, \quad \hat{y} = W_L ... W_2 W_1 x$$

The simplified network keeps only the effect we want to understand: repeated multiplication.

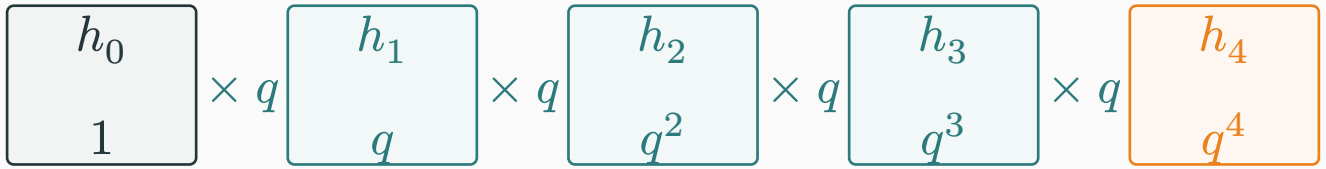
How one scalar activation changes through four layers

D DERIVATION

Here h_ℓ is the scalar activation after layer ℓ. Set the input activation to $h_0 = 1$.

Every layer multiplies its incoming activation by the same local gain q:

$$h_\ell = q h_{\ell-1} \quad \rightarrow \quad h_4 = q^4 h_0.$$



q	h_1	h_2	h_4
0.5000	0.5000	0.2500	0.0625
1.0000	1.0000	1.0000	1.0000
1.5000	1.5000	2.2500	5.0625

Backpropagation through the same four multiplications

D DERIVATION

Write $g_\ell = \partial \frac{\mathcal{L}}{\partial} h_\ell$. Suppose the gradient arriving at the output is $g_4 = 1$.

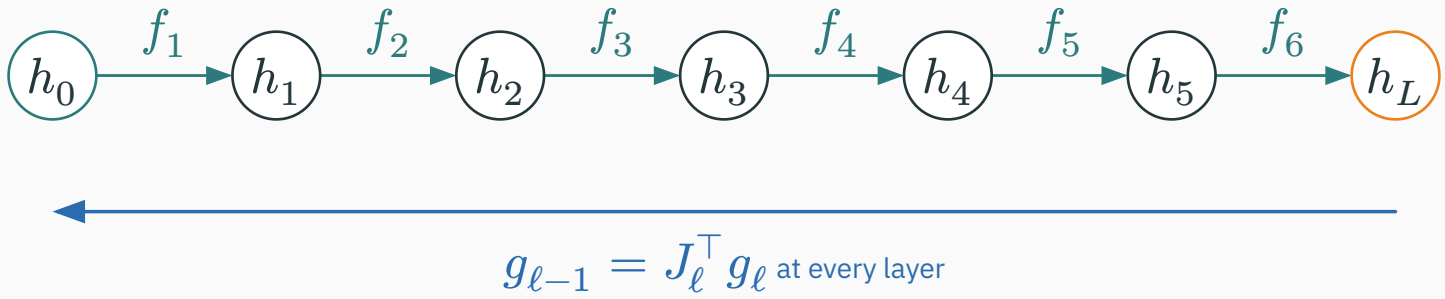
$$g_3 = q g_4, \quad g_2 = q g_3, \quad g_1 = q g_2, \quad g_0 = q g_1 = q^4.$$

local multiplier q	g_4	$g_2 = q^2$	$g_0 = q^4$
0.5	1	0.25	0.0625
1.0	1	1.0	1.0
1.5	1	2.25	5.0625

The same product that controls the forward signal also controls how much sensitivity reaches an early layer.

Matrix backpropagation multiplies transposed Jacobians

D DERIVATION



The schematic suppresses boldface; the equations below treat activations and gradients as vectors.

FORWARD

$$\boldsymbol{h}_\ell = f_{\ell(\boldsymbol{h}_{\ell-1})}$$
$$\boldsymbol{h}_L = f_L \bigcirc \ldots \bigcirc f_1(\boldsymbol{h}_0)$$

BACKWARD

$$\boldsymbol{g}_{\ell-1} = J_\ell^\top \boldsymbol{g}_\ell$$
$$\boldsymbol{g}_0 = J_1^\top \ldots J_L^\top \boldsymbol{g}_L$$

$J_\ell = \partial \frac{\boldsymbol{h}_\ell}{\partial} \boldsymbol{h}_{\ell-1}$. The scalar multiplier q was the one-dimensional version of this local Jacobian.

RMS measures the typical size of a gradient vector

D DERIVATION

A layer has many gradient coordinates, with positive and negative signs. Their ordinary mean can cancel.

ONE GRADIENT VECTOR

$$g = (2, -2, 1, -1)$$

Its mean is 0, but its coordinates are not small.

ROOT MEAN SQUARE

$$\mathrm{RMS}(g) = \sqrt{\frac{2^2+(-2)^2+1^2+(-1)^2}{4}}$$

$$= \sqrt{2.5} \approx 1.58$$

RMS ignores sign but preserves magnitude: it is the size of a typical coordinate in the vector.

One-layer RMS gain is a ratio of gradient sizes

D DERIVATION

Backpropagation maps $\boldsymbol{g}_\ell$ to $\boldsymbol{g}_{\ell-1} = J_\ell^\top \boldsymbol{g}_\ell$. Compare their RMS values.

Suppose a layerwise diagnostic reports the following hypothetical measurements on one batch.

ARRIVING FROM THE LATER LAYER $\leftarrow$ **REACHING THE EARLIER LAYER**

$$\mathrm{RMS}(\boldsymbol{g}_\ell) = 1.00 \qquad J_\ell^\top \qquad \mathrm{RMS}(\boldsymbol{g}_{\ell-1}) = 0.90$$

$$\rho_\ell := \frac{\mathrm{RMS}(\boldsymbol{g}_{\ell-1})}{\mathrm{RMS}(\boldsymbol{g}_\ell)} = \frac{0.90}{1.00} = 0.90$$

This layer reduced the typical gradient coordinate by 10%.

Input-gradient RMS is multiplied by $\prod_{\ell=1}^{30} \rho_\ell$

D DERIVATION

By the definition of each ratio, the product telescopes exactly:

$$\mathrm{RMS}(\boldsymbol{g}_0) = \prod_{\ell=1}^{30} \rho_\ell \cdot \mathrm{RMS}(\boldsymbol{g}_{30}).$$

If all measured gains are close to one value ρ, the multiplier is approximately ρ^{30}.

$\rho = 0.9$	$\rho = 1.0$	$\rho = 1.1$
$0.9^{30} \approx 0.042$	$1^{30} = 1$	$1.1^{30} \approx 17.45$
only 4.2% reaches the start	scale is preserved	over $17\times$ reaches the start

Vanishing gradients starve early layers; exploding signals destabilize numerics

V VISUAL

PLAIN SGD UPDATE FOR THE FIRST LAYER

$$\frac{\partial \mathcal{L}}{\partial W_1} = \boldsymbol{g}_1 \boldsymbol{h}_0^\top, \quad W_1 \leftarrow W_1 - \eta \frac{\partial \mathcal{L}}{\partial W_1}.$$

Adam rescales coordinate updates, but it cannot recover zero or underflowed gradients or repair badly scaled forward activations.

VANISHING

$\mathrm{RMS}(\boldsymbol{g}_1) \approx 0$

The early weights barely change; features near the input learn very slowly.

USABLE SCALE

$\mathrm{RMS}(\boldsymbol{g}_1)$ is neither tiny nor huge and is comparable across depth

The optimizer receives a meaningful direction and scale.

EXPLODING

$\mathrm{RMS}(\boldsymbol{g}_1)$ grows rapidly toward the input

SGD steps can become huge; forward or backward values may overflow or be clipped.

Initialization: break symmetry and control signal scale

Random initialization has two jobs

V VISUAL

JOB 1 · BREAK SYMMETRY

Hidden units must begin differently.

Independent random values give them different features and different gradients.

JOB 2 · CONTROL SCALE

Signals must survive depth.

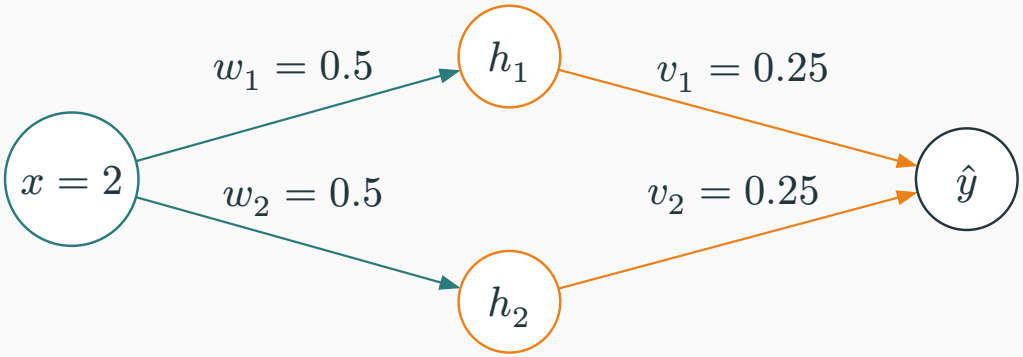
The weight variance should keep forward activations and backward gradients usable.

First break symmetry; then calculate the variance.

Exact clones receive identical updates and remain clones

D DERIVATION

Exact means equal incoming and outgoing parameters, optimizer state, and deterministic operations.



Take $h_i = \mathrm{ReLU}(w_i x)$, $\hat{y} = v_1 h_1 + v_2 h_2$, and $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$ with

$$w_1 = w_2 = 0.5, \quad v_1 = v_2 = 0.25, \quad x = 2, \quad y = 1.$$

FORWARD

$$h_1 = h_2 = 1$$
$$\hat{y} = 0.5$$

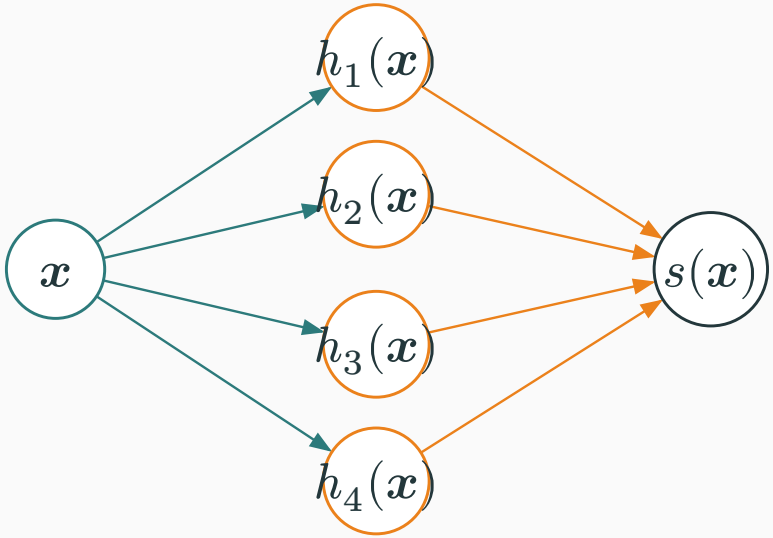
BACKWARD

$$\partial \frac{\mathcal{L}}{\partial} v_1 = \partial \frac{\mathcal{L}}{\partial} v_2 = -0.5$$
$$\partial \frac{\mathcal{L}}{\partial} w_1 = \partial \frac{\mathcal{L}}{\partial} w_2 = -0.25$$

Four cloned neurons still provide only one feature

V VISUAL

INSIDE THE HIDDEN LAYER



IF ALL FOUR COPIES STAY EQUAL

$$h_{j(x)} = h(x) \text{ and } v_j = v \text{ for every } j \in \{1, \ldots, 4\}.$$

$$s(x) = \sum_{j=1}^{4} v h_{j(x)} + c$$

$$= 4vh(x) + c$$

The output can rescale one feature, but it cannot create three new feature directions.

LAYER WIDTH

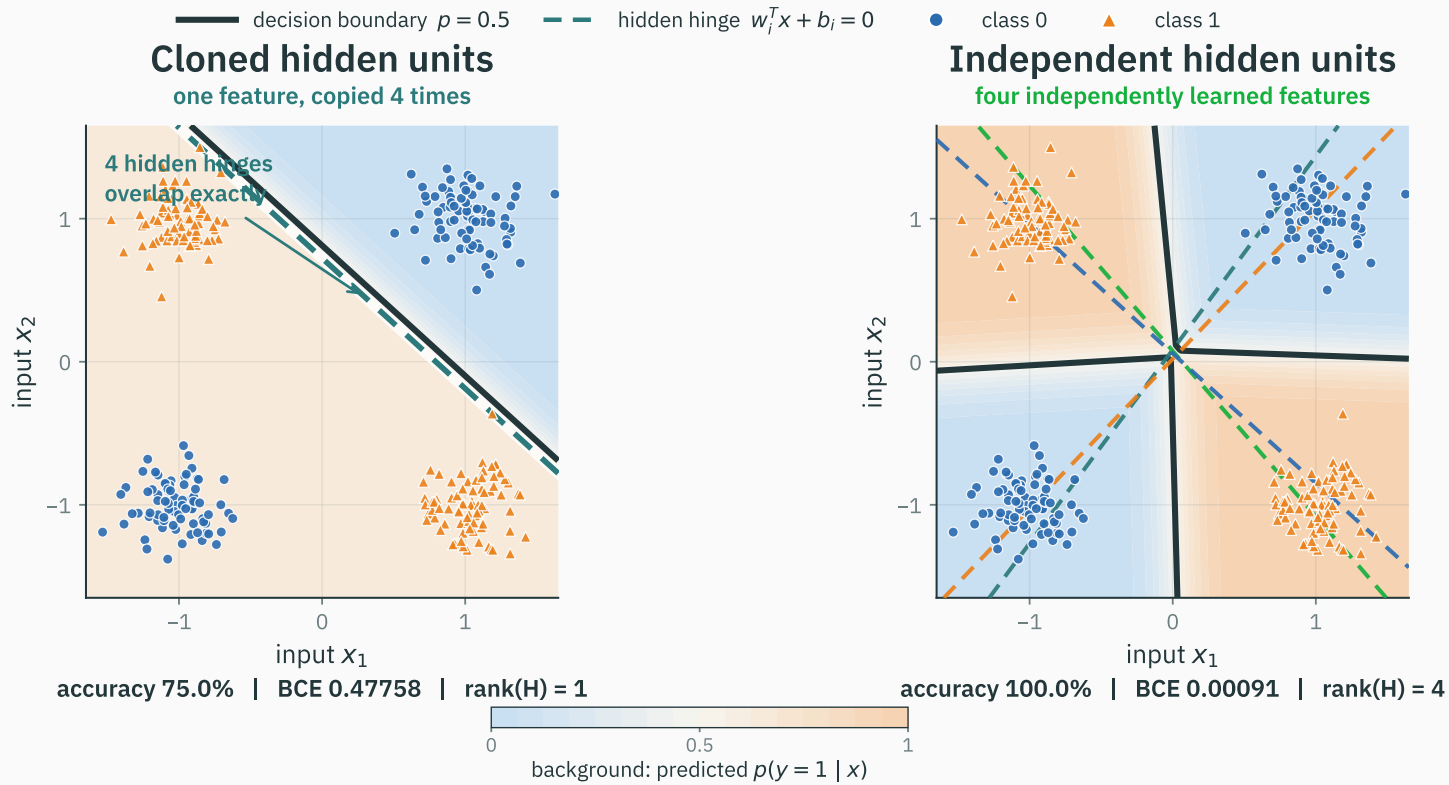
4 neurons

DISTINCT HIDDEN FEATURES

1 feature

XOR: cloned units reach 75%; independent units reach 100%

V VISUAL



Same 320-point XOR data, $2 \to 4$ ReLU $\to 1$ model, full-batch Adam, and 300 updates. $\mathrm{rank}(H)$ is the rank of the 320×4 hidden-activation matrix.

Interactive: when do cloned neurons separate?

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/symmetry-breaking/

Open the **hidden-unit symmetry lab**. Keep the data and architecture fixed; change only how the four hidden units start.

1 · EXACTLY CLONED

Click **one step**. The four hinge lines remain on top of one another and parameter separation stays zero.

2 · SMALL PERTURBATION

Nudge the initial copies apart. Their gradients differ and the hinge lines can begin to specialize.

3 · INDEPENDENT

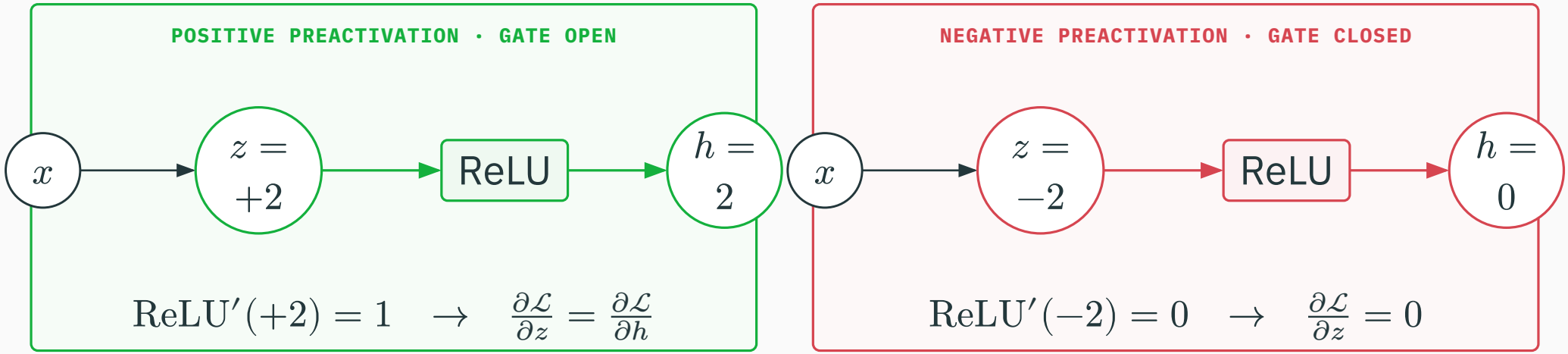
Run training. Compare accuracy, loss, hidden-feature rank, and the learned decision surface.

Exact cloning preserves the symmetry. A distinct start lets the units learn distinct features.

ReLU passes positive preactivations and blocks negative ones

V VISUAL

$$z = w^\top x + b, \quad h = \mathrm{ReLU}(z) = \max(0, z)$$



A negative preactivation is blocked in both the forward pass and the backward pass.

The gate decides whether this example updates w and b

D DERIVATION

Suppose the gradient arriving at the ReLU output is $\partial \frac{\mathcal{L}}{\partial} h = 4$, with $z = wx + b$ and $x = 2$.

CLOSED GATE

$$w = -1, \quad b = -1$$
$$z = (-1)(2) - 1 = -3$$
$$h = \mathrm{ReLU}(-3) = 0$$

$$\frac{\partial \mathcal{L}}{\partial z} = 4 \times 0 = 0$$
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} x = 0$$
$$\frac{\partial \mathcal{L}}{\partial b} = 0$$

OPEN GATE

$$w = 1, \quad b = -1$$
$$z = (1)(2) - 1 = 1$$
$$h = \mathrm{ReLU}(1) = 1$$

$$\frac{\partial \mathcal{L}}{\partial z} = 4 \times 1 = 4$$
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} x = 8$$
$$\frac{\partial \mathcal{L}}{\partial b} = 4$$

The same example updates the incoming parameters only when its ReLU gate is open.

A neuron is dead on the training set when no example opens it

V VISUAL

Take three inputs $x \in \{-1, 0, 1\}$ and $z = wx + b$ with $w = 1$. Suppose an update moves the bias from -0.2 to -2.

case	$x = -1$	$x = 0$	$x = 1$	result over all three examples
before $b = -0.2$	$z = -1.2$ $h = 0$	$z = -0.2$ $h = 0$	$z = 0.8$ $h = 0.8$	**alive** one gate opens
after $b = -2$	$z = -3$ $h = 0$	$z = -2$ $h = 0$	$z = -1$ $h = 0$	**dead** all three gates closed

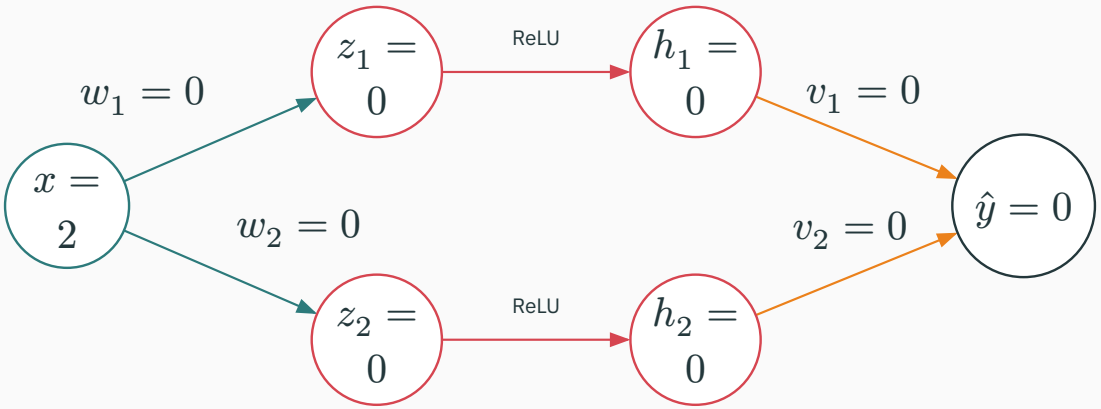
Some negative preactivations are normal. If every preactivation is negative, this neuron's data gradient is zero.

All-zero initialization puts every hidden ReLU at $z = 0$

V VISUAL

In the same two-unit network, set every hidden bias and every weight to zero:

$$w_1 = w_2 = b_1 = b_2 = v_1 = v_2 = 0, \quad x = 2, \quad y = 1.$$



$z_1 = z_2 = 0$ and $h_1 = h_2 = \mathrm{ReLU}(0) = 0$. At this kink the derivative is undefined mathematically; PyTorch uses $\mathrm{ReLU}'(0) = 0$.

Predict before calculating: can either layer receive a nonzero data gradient?

With every weight zero, both weight gradients are zero

D DERIVATION

For $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$, the output derivative is $\partial \frac{\mathcal{L}}{\partial} \hat{y} = \hat{y} - y = -1$.

OUTPUT WEIGHTS

$$\frac{\partial \mathcal{L}}{\partial v_i} = (\hat{y} - y) h_i$$

$$= (-1)(0) = 0$$

The output weights cannot move because every hidden activation is zero.

HIDDEN WEIGHTS

$$\frac{\partial \mathcal{L}}{\partial w_i} = (\hat{y} - y) v_i \mathrm{ReLU}'(0) x$$

$$= (-1)(0)(0)(2) = 0$$

The hidden weights cannot move because the outgoing path is also zero.

An output bias, if present, can learn a constant; the hidden features still remain zero.

$w_i^+ = v_i^+ = 0$**: the same state repeats. All-zero initialization is stuck; hidden units must not begin as exact clones. Zero biases are fine.**

Choosing the weight scale

Different nonzero weights can still have the wrong scale

V VISUAL

ALREADY FIXED · SYMMETRY

Start neurons differently.

Independent nonzero weights let hidden units learn different features.

REMAINING QUESTION · SCALE

How large should the weights be?

Their standard deviation sets how much one layer expands or shrinks a signal.

WEIGHT SCALE

$$\mathrm{Std}(w)$$

AFFINE-MAP GAIN

$$s_\ell := \frac{\mathrm{RMS}(z_\ell)}{\mathrm{RMS}(h_{\ell-1})}$$

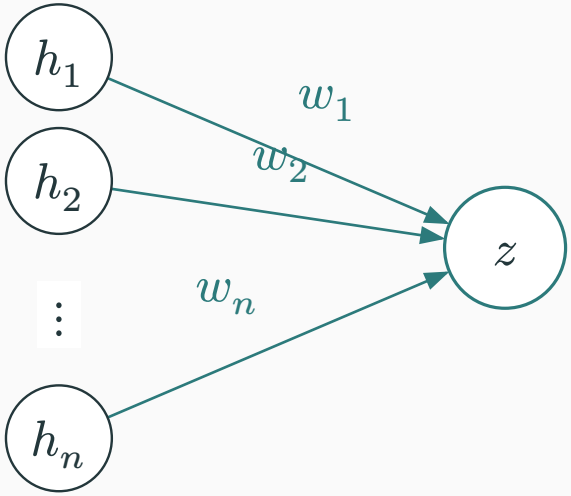
FULL-LAYER GAIN

$$\kappa_\ell := \frac{\mathrm{RMS}(h_\ell)}{\mathrm{RMS}(h_{\ell-1})}$$

$\frac{\mathrm{RMS}(h_L)}{\mathrm{RMS}(h_0)} = \prod_{\ell=1}^{L} \kappa_\ell$. **At initialization, the target is** $\kappa_\ell \approx 1$.

One neuron sums one weighted contribution from each input

V VISUAL



ONE LINEAR PREACTIVATION

$u_i = w_i h_i$ is one weighted contribution.

$$z = u_1 + u_2 + \ldots + u_n = \sum_{i=1}^{n} w_i h_i$$

fan-in $= n$: the number of values entering the neuron.

Why calculate this? z is passed into the activation; an RMS gain here compounds through depth.

Prediction: for 100 independent, zero-mean terms with RMS 1, is $\mathrm{RMS}(z)$ closer to 1, 10, or 100?

Two independent ± 1 terms give a sum with RMS $\sqrt{2}$

D DERIVATION

Each term is independently -1 or $+1$ with probability $\frac{1}{2}$. Thus $\mathbb{E}[u_i] = 0$ and $\mathrm{RMS}(u_i) = 1$.

u_1	u_2	$z = u_1 + u_2$	z^2
-1	-1	-2	4
-1	$+1$	0	0
$+1$	-1	0	0
$+1$	$+1$	2	4

AVERAGE SQUARED SUM

$$\mathbb{E}[z^2] = \frac{4+0+0+4}{4} = 2$$

$$\mathrm{RMS}(z) = \sqrt{2}$$

$$z^2 = u_1^2 + u_2^2 + 2u_1u_2$$

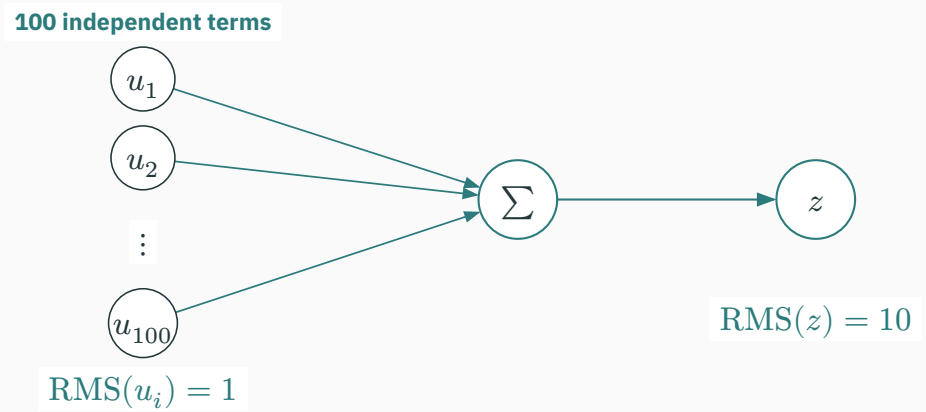
$$u_1u_2 = [+1, -1, -1, +1]$$

$$\mathbb{E}[u_1u_2] = 0$$

The cross term averages to zero. Squared sizes add first; take the square root afterward.

With 100 independent zero-mean, unit-RMS terms, $\mathrm{RMS}(z) = 10$

D DERIVATION



1 TERM $\mathrm{RMS}(z) = 1$	4 TERMS $\mathrm{RMS}(z) = 2$	100 TERMS $\mathrm{RMS}(z) = 10$

By the same calculation, $\mathbb{E}[z^2] = 100$ and $\mathrm{RMS}(z) = \sqrt{100} = 10$.

This RMS is over repeated random draws; one realized 100-term sum need not equal 10.

Without fan-in scaling, width changes the one-layer gain, and depth multiplies those gains.

With fan-in 100, $\mathrm{Std}(w) = 0.1$ preserves linear RMS

D DERIVATION

Assume the incoming coordinates have RMS 1. Compare two random initializations of the same neuron.

UNSCALED WEIGHTS

$$\mathrm{Std}(w) = 1$$
$$\mathrm{RMS}(w_i h_i) = 1$$
$$\mathrm{RMS}(z) = \sqrt{100} \times 1 = 10$$

FAN-IN SCALED WEIGHTS

$$\mathrm{Std}(w) = 0.1$$
$$\mathrm{RMS}(w_i h_i) = 0.1$$
$$\mathrm{RMS}(z) = \sqrt{100} \times 0.1 = 1$$

For 100 inputs, $\mathrm{Std}(w) = \frac{1}{\sqrt{100}} = 0.1$ and $\mathrm{Var}(w) = \frac{1}{100} = 0.01$.

This corrects the linear sum. We will account for the following ReLU separately.

For fan-in n, variance $\frac{1}{n}$ preserves the linear second moment

D DERIVATION

Assume $b = 0$; the weights are independent, zero-mean, independent of the incoming activations, and have common variance σ_w^2. The incoming coordinates share the second moment $\mathbb{E}[h_i^2] = \mathbb{E}[h^2]$.

$$\mathbb{E}[z^2] = \sum_{i=1}^{n} \mathbb{E}[w_i^2 h_i^2] = n\sigma_w^2 \mathbb{E}[h^2].$$

The cross terms vanish in expectation because the weights are independent and zero-mean.

$$\frac{\mathrm{RMS}(z)}{\mathrm{RMS}(h)} = \sqrt{n\ \mathrm{Var}(w)}.$$

VARIANCE

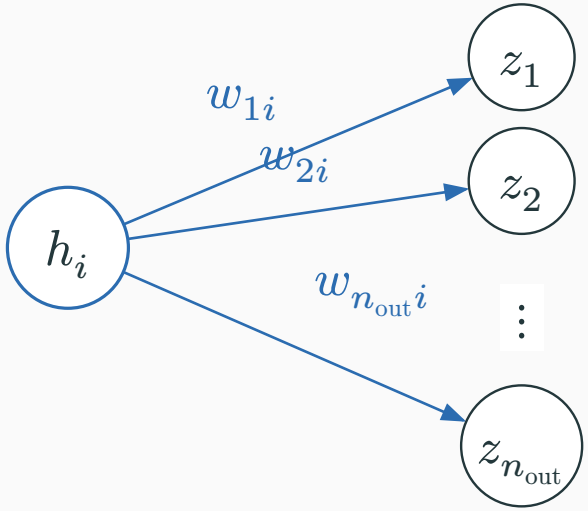
$$\mathrm{Var}(w) = \frac{1}{n}$$

STANDARD DEVIATION

$$\mathrm{Std}(w) = \frac{1}{\sqrt{n}}$$

An affine map backpropagates a fan-out weighted sum

D DERIVATION



ONE INCOMING ACTIVATION

It influences n_{out} preactivations.
fan-out $= n_{\text{out}}$: the number of values this activation feeds.

$$\partial \frac{\mathcal{L}}{\partial} h_i = \sum_{j=1}^{n_{\text{out}}} w_{ji} \partial \frac{\mathcal{L}}{\partial} z_j$$

FORWARD

At initialization, independent zero-mean contributions add; $\mathrm{Var}(w) \approx \frac{1}{n_{\text{in}}}$ preserves the affine second moment.

BACKWARD

Similarly, $\mathrm{Var}(w) \approx \frac{1}{n_{\text{out}}}$ preserves the affine backward second moment. Activation derivatives are separate.

For identity and near-linear tanh, Xavier balances fan-in and fan-out

D DERIVATION

FORWARD PREFERS

$$\frac{1}{n_{\text{in}}}$$

VS

BACKWARD PREFERS

$$\frac{1}{n_{\text{out}}}$$

When $n_{\text{in}} \neq n_{\text{out}}$, one variance cannot satisfy both exactly.

XAVIER / GLOROT COMPROMISE

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

For a square layer, $n_{\text{in}} = n_{\text{out}} = n$, so Xavier reduces to $\text{Var}(W) = \frac{1}{n}$.

ReLU keeps half the second moment of a symmetric input

D DERIVATION

BEFORE RELU

$$z = [-2, -1, 1, 2]$$
$$\text{mean}(z^2) = \tfrac{10}{4} = 2.5$$

$\rightarrow$

AFTER RELU

$$h = [0, 0, 1, 2]$$
$$\text{mean}(h^2) = \tfrac{5}{4} = 1.25$$

For a distribution symmetric around zero, $\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2}\mathbb{E}[z^2]$**.**

The affine map must double the second moment before ReLU halves it.

For ReLU, He/Kaiming uses $\mathrm{Std}(w) = \sqrt{\frac{2}{\text{fan-in}}}$

D DERIVATION

At initialization, assume independent zero-mean weights, zero bias, and an approximately symmetric zero-centred preactivation.

$$\mathbb{E}[h_{\text{next}}^2] \approx \frac{1}{2} n_{\text{in}} \, \mathrm{Var}(w) \mathbb{E}[h^2].$$

Set the per-layer second-moment gain to one:

$$\frac{1}{2} n_{\text{in}} \, \mathrm{Var}(w) = 1 \quad \rightarrow \quad \mathrm{Var}(w) = \frac{2}{n_{\text{in}}}, \quad \mathrm{Std}(w) = \sqrt{\frac{2}{\text{fan-in}}}.$$

WORKED FAN-IN 100

$$\mathrm{Std}_{\mathrm{He}} = \sqrt{\tfrac{2}{100}} \approx 0.141$$

OUR HIDDEN WIDTH 128

$$\mathrm{Std}_{\mathrm{He}} = \sqrt{\tfrac{2}{128}} = 0.125$$

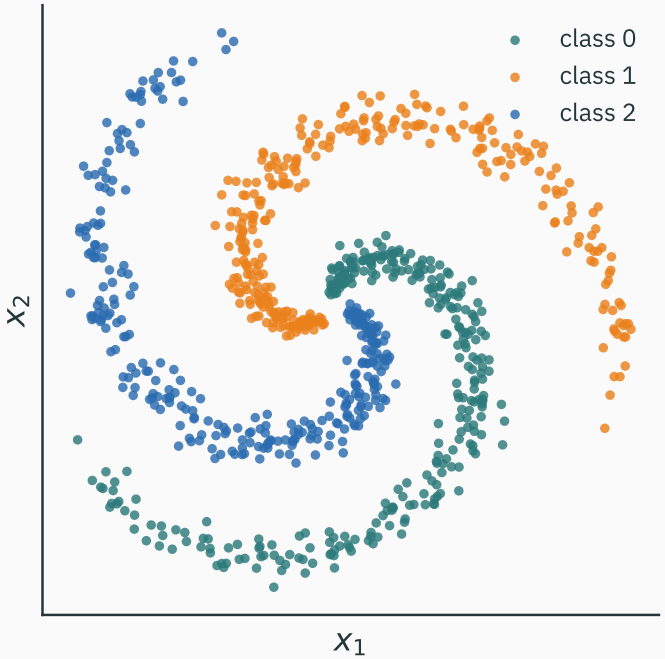
PyTorch: `nn.init.kaiming_normal_(W, mode="fan_in", nonlinearity="relu")`

This stabilizes scale at initialization; it does not guarantee that every deep network will train.

Test the calculation on a 30-hidden-layer MLP

A 30-hidden-layer MLP on a 2-D spiral

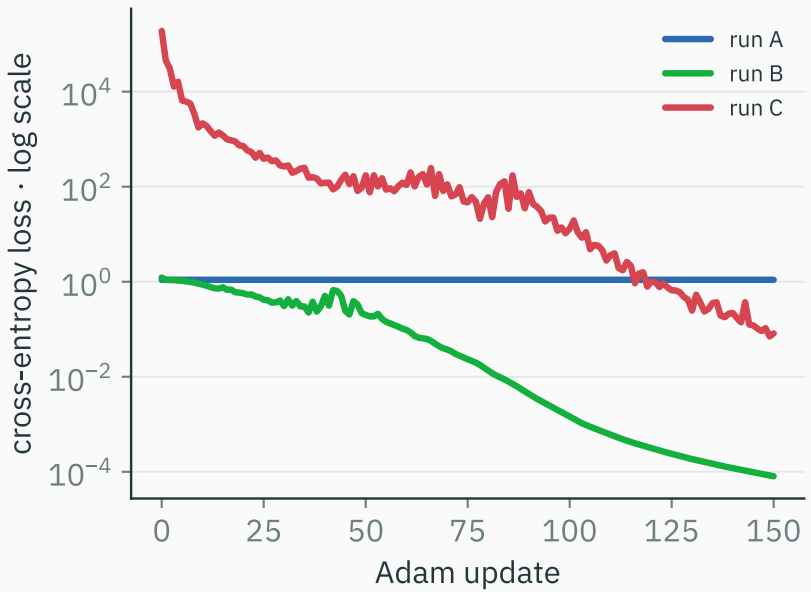
V VISUAL




```
model = nn.Sequential(
    nn.Linear(2, 128), nn.ReLU(),
    # ... 29 more hidden Linear-ReLU blocks ...
    nn.Linear(128, 3),
)
opt = torch.optim.Adam(
    model.parameters(), lr=3e-4
)
```

900 points · 3 classes
30 hidden layers · ReLU
cross-entropy · full-batch Adam

Training loss and accuracy for runs A–C



One run is stuck; two learn at different rates. What single setting changed?

What changed between runs A, B, and C?

What was the only thing changed between runs A, B, and C?

A. optimizer

B. learning rate

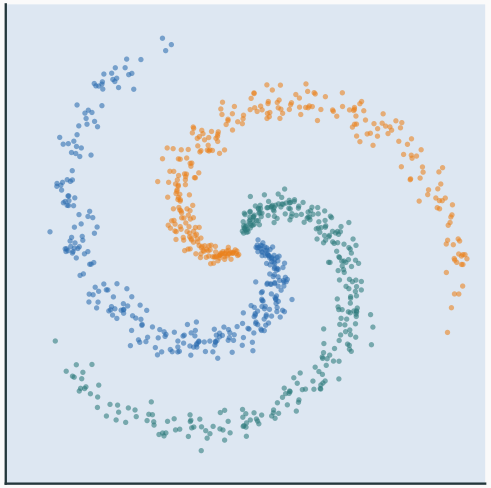
C. initial weight scale

D. network depth

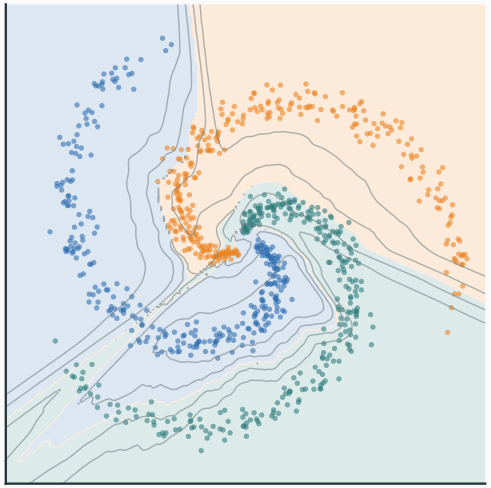
Decision regions after 50 Adam updates

after 50 Adam updates · data and optimizer fixed

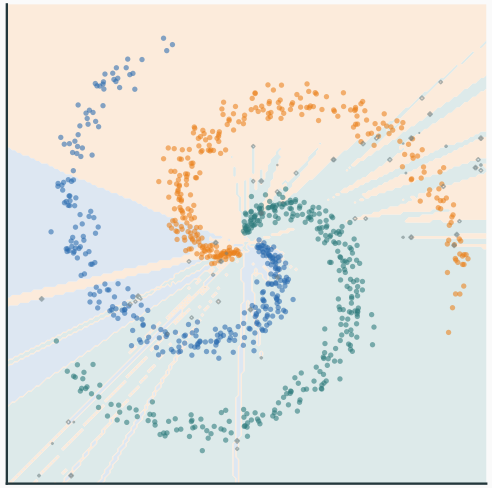
$\alpha = 0.5$ · accuracy 33.3%



$\alpha = 1.0$ · accuracy 93.2%



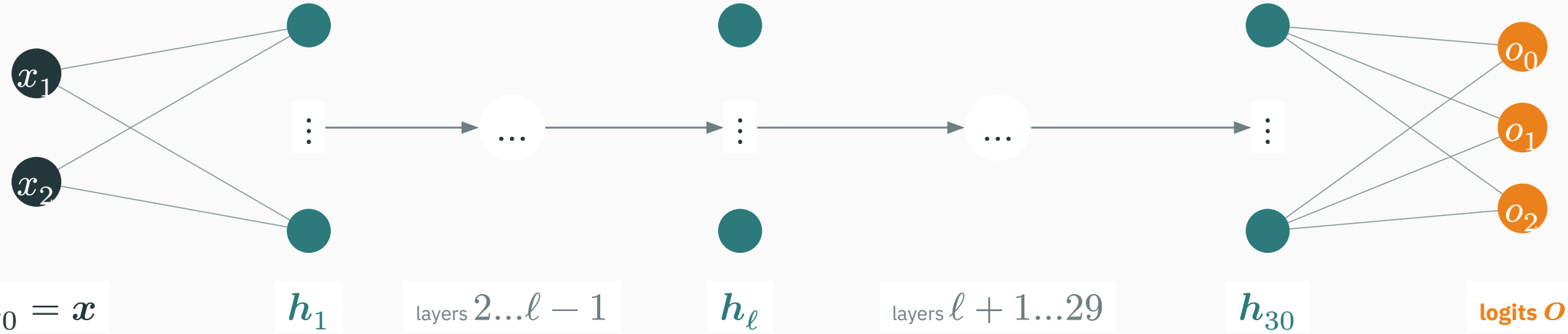
$\alpha = 1.5$ · accuracy 70.3%



Only α changes across the three runs. What quantity do you think α controls?

Network and tensor notation for the 30-hidden-layer MLP

V VISUAL



INPUT

$$h_0 = x \in \mathbb{R}^2$$

HIDDEN LAYERS $\ell = 1, \ldots, 30$

$$z_\ell = W_\ell h_{\ell-1} + b_\ell$$
$$h_\ell = \mathrm{ReLU}(z_\ell) \in \mathbb{R}^{128}$$

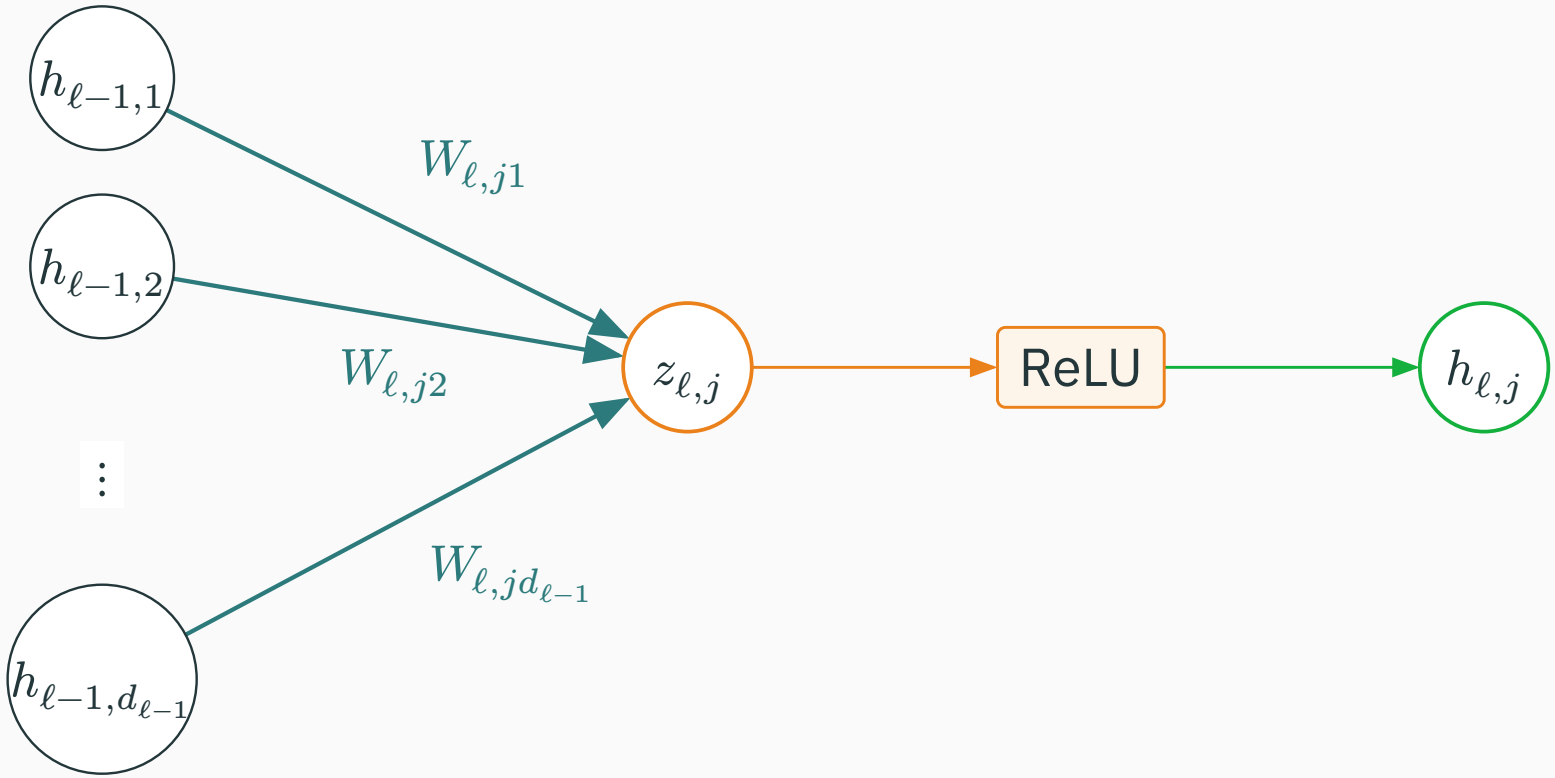
OUTPUT LAYER 31

$$o = W_{31} h_{30} + b_{31} \in \mathbb{R}^3$$

No ReLU on the logits.

The subscript ℓ names a hidden layer. h_ℓ is the vector that leaves its ReLU and enters the next layer.

Inside hidden layer ℓ: one neuron and its weights



$$z_{\ell,j} = \sum_{i=1}^{d_{\ell-1}} W_{\ell,ji} h_{\ell-1,i} + b_{\ell,j} \quad h_{\ell,j} = \max(0, z_{\ell,j})$$

Read $W_{\ell,ji}$ as layer ℓ, row j, column i

D DERIVATION

ℓ · LAYER

Which hidden layer: $1, \ldots, 30$.

i · INCOMING COORDINATE

Which value in $\boldsymbol{h}_{\ell-1}$.

j · RECEIVING NEURON

Which weighted sum in layer ℓ.

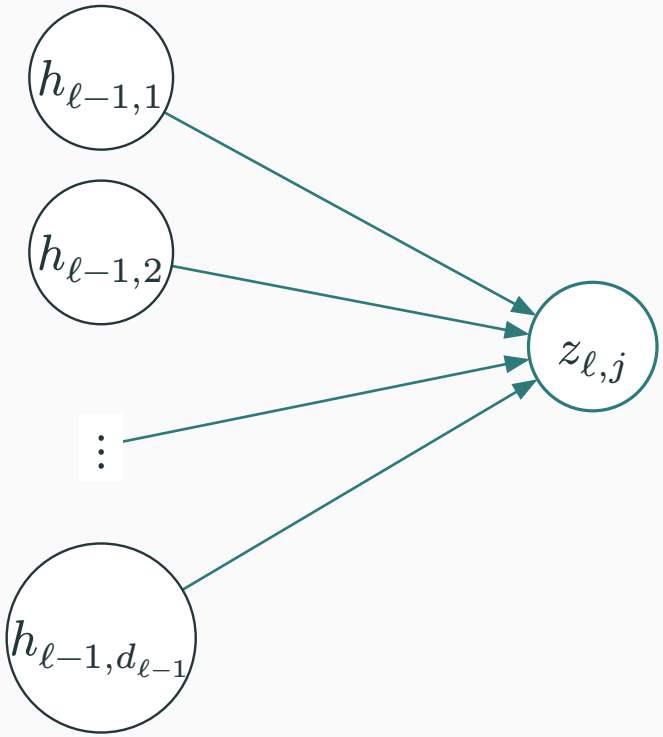
CONCRETE EXAMPLE

$$h_{4,3} \xrightarrow{W_{5,73}} z_{5,7}$$

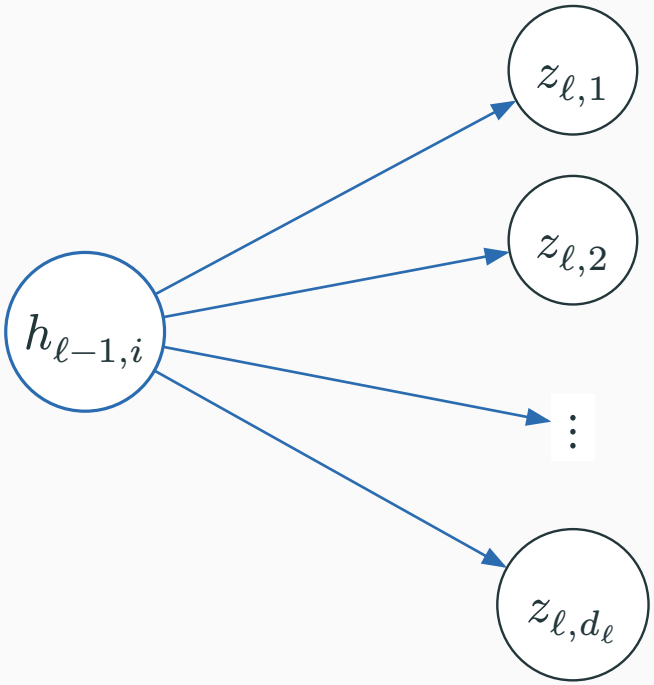
The entry $W_{5,73}$ multiplies input coordinate 3 when neuron 7 in layer 5 forms its weighted sum.

For W_ℓ: rows use fan-in; columns use fan-out

FAN-IN OF ONE OUTPUT NEURON



FAN-OUT OF ONE INPUT VALUE



$$\boldsymbol{z}_\ell = W_\ell \boldsymbol{h}_{\ell-1} + \boldsymbol{b}_\ell, \quad W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$$

Fan-in and fan-out in this MLP

W_1 · INPUT TO HIDDEN 1

$$2 \to 128$$

$$W_1 \in \mathbb{R}^{128 \times 2}$$

fan-in $= 2$
fan-out $= 128$

$W_2, \ldots, W_{30}$ · HIDDEN

$$128 \to 128$$

$$W_\ell \in \mathbb{R}^{128 \times 128}$$

fan-in $= 128$
fan-out $= 128$

W_{31} · HIDDEN 30 TO OUTPUT

$$128 \to 3$$

$$W_{31} \in \mathbb{R}^{3 \times 128}$$

fan-in $= 128$
fan-out $= 3$

In PyTorch, `nn.Linear(d_in, d_out).weight.shape` is `(d_out, d_in)`.

He/Kaiming standard deviations in this MLP

D DERIVATION

One preactivation is a sum of $d_{\ell-1}$ incoming contributions: $z_{\ell,j} = \sum_i W_{\ell,ji} h_{\ell-1,i}$.

1 · THE SUM

Independent contributions add their variances.

$$\mathrm{Var}(z_{\ell,j}) \approx d_{\ell-1}\,\mathrm{Var}(W_{\ell,ji})\,\mathbb{E}\left[h_{\ell-1,i}^2\right]$$

→

2 · CANCEL THE WIDTH

Choose weight variance proportional to $\frac{1}{d_{\ell-1}}$.

A wider layer should not create a larger z_ℓ.

→

3 · COMPENSATE FOR RELU

At symmetric initialization, ReLU removes roughly half the squared signal.

Multiply the variance by 2.

Hidden-layer He/Kaiming normal: $W_{\ell,ji} \sim \mathcal{N}\left(0, \frac{2}{d_{\ell-1}}\right)$**, so** $\mathrm{Std}(W_{\ell,ji}) = \sqrt{\frac{2}{\text{fan-in}}}$**.**

Only α changes across runs A, B, and C


```
with torch.no_grad():
    nn.init.kaiming_normal_(W, mode="fan_in", nonlinearity="relu")
    W.mul_(alpha)   # 0.5, 1.0, or 1.5
```

$\alpha = 0.5$
half the He standard deviation

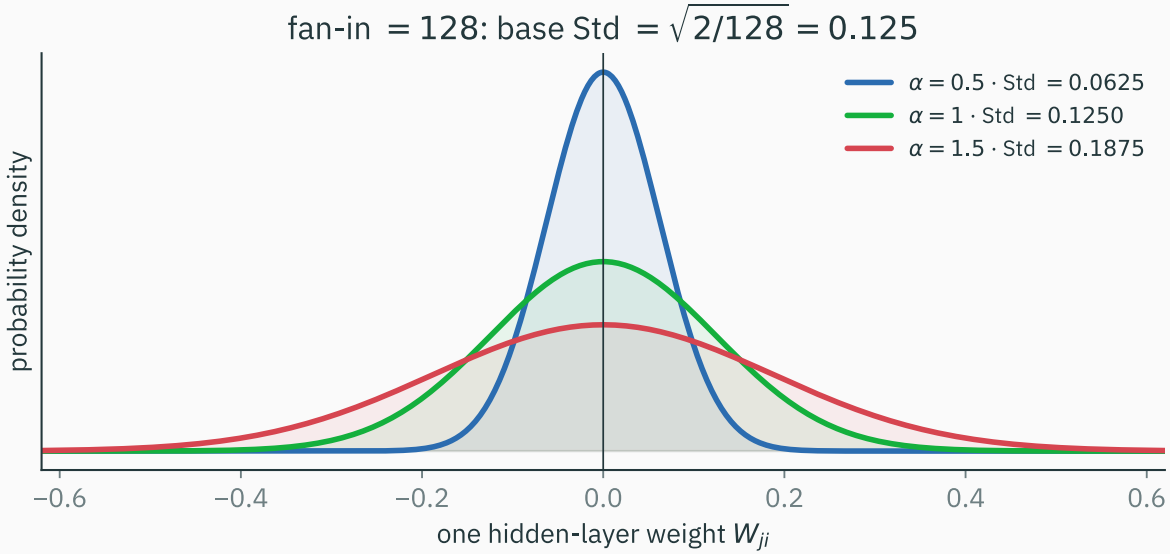
$\alpha = 1$
ordinary He initialization

$\alpha = 1.5$
$1.5 \times$ the He standard deviation

The random seed is reset, so every run uses the same weight directions and signs. Only their common scale changes.

Weight standard deviations for $\alpha = 0.5$, 1.0, and 1.5

$$\varepsilon_{\ell,ji} \sim \mathcal{N}(0,1), \quad W_{\ell,ji}^{(\mathrm{He})} = \sqrt{\frac{2}{d_{\ell-1}}}\varepsilon_{\ell,ji}, \quad W_{\ell,ji}^{(\alpha)} = \alpha W_{\ell,ji}^{(\mathrm{He})}$$

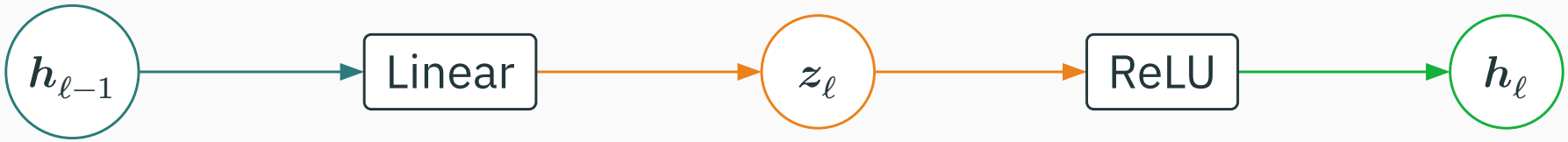


For hidden blocks $W_2, \ldots, W_{30}$ with fan-in 128, the He standard deviation is $\sqrt{\frac{2}{128}} = 0.125$. The three curves therefore have standard deviations 0.0625, 0.125, and 0.1875.

The hidden layers use He initialization for ReLU. For this controlled experiment, W_{31} uses the same rule even though the logits are not followed by ReLU; this is an experimental control, not a general output-layer recommendation.

Recorded post-ReLU tensor: $H_\ell \in \mathbb{R}^{900\times128}$

V VISUAL



THE BATCH TENSOR H_ℓ

	unit 1	unit 2	…	unit 128
example 1	$h_{\ell,1}^{(1)}$	$h_{\ell,2}^{(1)}$	…	$h_{\ell,128}^{(1)}$
example s	$h_{\ell,1}^{(s)}$	$h_{\ell,2}^{(s)}$	…	$h_{\ell,128}^{(s)}$
example 900	$h_{\ell,1}^{(900)}$	$h_{\ell,2}^{(900)}$	…	$h_{\ell,128}^{(900)}$

$$\boldsymbol{h}_\ell^{(s)} = H_{\ell[s,:]} \in \mathbb{R}^{128}, \quad H_\ell \in \mathbb{R}^{900\times128}$$

Rows are examples; columns are hidden units.

READ THE NOTATION

ℓ chooses the hidden layer.

s chooses one example, hence one row.

j chooses one hidden unit, hence one column.

WHERE TO OBSERVE

Linear output $\longrightarrow \boldsymbol{z}_\ell$

ReLU output $\longrightarrow \boldsymbol{h}_\ell$

This lecture records $\boldsymbol{h}_\ell$.

Hooks tutorial: activations + gradients ↗

RMS measures the typical size of one coordinate

$$\mathrm{RMS}(t) = \sqrt{\mathrm{mean}(t^2)}$$

FOUR COORDINATES

$$t = [3, 4, 0, 0]$$

$$\|t\|_2 = \sqrt{25} = 5$$
$$\mathrm{RMS}(t) = \sqrt{\tfrac{25}{4}} = 2.5$$

SAME VALUES, REPEATED TWICE

$$t' = [3, 4, 0, 0, 3, 4, 0, 0]$$

$$\|t'\|_2 = \sqrt{50} = 7.07$$
$$\mathrm{RMS}(t') = \sqrt{\tfrac{50}{8}} = 2.5$$

Repeating the same values increases the norm, even though no coordinate became larger. RMS remains 2.5.

For H_ℓ, the mean is over all 900×128 entries. RMS is a diagnostic summary, not part of the loss.

Four checks for every hidden layer

Here $\mathcal{L}$ is mean cross-entropy over all 900 examples; $H_0 = X$ and $G_0 = \partial \frac{\mathcal{L}}{\partial} X$ are 900×2.

FORWARD PASS

1 · activation RMS

$$\sqrt{\text{mean}(H_\ell^2)}$$

Typical magnitude of a post-ReLU activation.

2 · active-ReLU fraction

$$\text{mean}(H_\ell > 0)$$

Fraction of activations that passed the ReLU gate.

3 · finite values

$$\text{all isfinite}(H_\ell)$$

Checks the activations for NaN or infinity.

AFTER BACKPROPAGATION

$$G_\ell = \partial \frac{\mathcal{L}}{\partial} H_\ell$$

For $\ell = 1, \ldots, 30$, H_ℓ and G_ℓ both have shape 900×128.

4 · activation-gradient RMS

$$\sqrt{\text{mean}(G_\ell^2)}$$

Typical sensitivity of the loss to the layer's activations.

Read the trace from the input: gradual drift suggests repeated scale mismatch; a sudden break identifies a layer to inspect.

For $\alpha > 0$, $\mathrm{ReLU}(\alpha z) = \alpha\ \mathrm{ReLU}(z)$

DIAGNOSTIC 1 · ACTIVATION RMS

$$h_0 = x, \quad W_\ell^{(\alpha)} = \alpha W_\ell^{(1)}, \quad b_\ell = 0, \quad \alpha > 0$$

IF $z < 0$

Example: $z = -2, \alpha = 0.5$
$\mathrm{ReLU}(\alpha z) = \mathrm{ReLU}(-1) = 0 = \alpha\ \mathrm{ReLU}(-2)$

IF $z > 0$

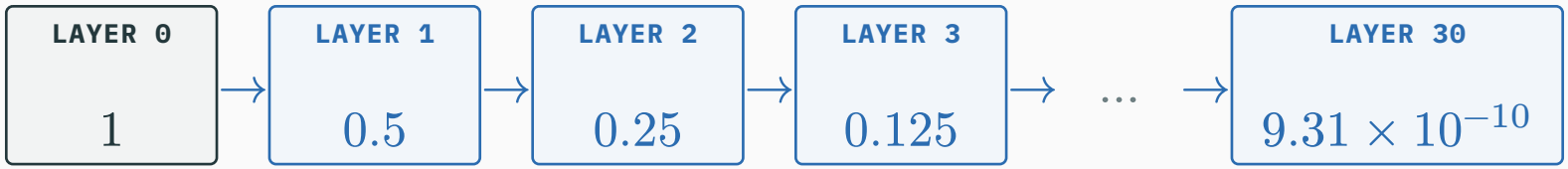
Example: $z = 3, \alpha = 0.5$
$\mathrm{ReLU}(\alpha z) = \mathrm{ReLU}(1.5) = 1.5 = \alpha\ \mathrm{ReLU}(3)$

For a positive scale, ReLU preserves the scale: $\mathrm{ReLU}(\alpha z) = \alpha\ \mathrm{ReLU}(z)$**.**

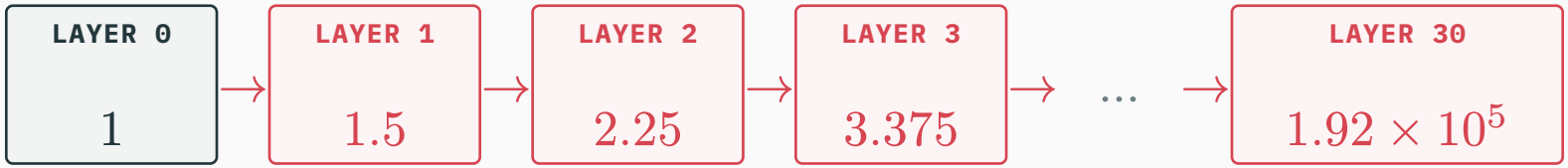
Relative signal scale after successive hidden layers

The $\alpha = 1$ run is the reference. Start its relative scale at 1 and apply α once per hidden layer.

$\alpha = 0.5$ · **EACH LAYER HALVES THE RELATIVE SCALE**



$\alpha = 1.5$ · **EACH LAYER MULTIPLIES THE RELATIVE SCALE BY** 1.5



After ℓ hidden layers, the relative scale is α^{ℓ}.

Batch activation RMS scales as α^ℓ

D DERIVATION

LAYER 1 · ONE FACTOR

$$\boldsymbol{h}_1^{(\alpha)} = \alpha \boldsymbol{h}_1^{(1)}$$

LAYER 2 · TWO FACTORS

$$\boldsymbol{h}_2^{(\alpha)} = \alpha^2 \boldsymbol{h}_2^{(1)}$$

LAYER ℓ · ℓ FACTORS

$$\boldsymbol{h}_\ell^{(\alpha)} = \alpha^\ell \boldsymbol{h}_\ell^{(1)}$$

STACK ALL 900 EXAMPLES

$$H_\ell^{(\alpha)} = \alpha^\ell H_\ell^{(1)}$$

Every entry is multiplied by the same positive number.

$\rightarrow$

THEREFORE THE RMS SCALES TOO

$$\mathrm{RMS}\left(H_\ell^{(\alpha)}\right) = \alpha^\ell \, \mathrm{RMS}\left(H_\ell^{(1)}\right)$$

$\mathrm{RMS}(cT) = |c|\mathrm{RMS}(T)$ and $\alpha > 0$.

This is the bridge from the per-example activation $h_\ell^{(s)}$ to the batch measurement plotted in the experiment.

Measure the layer-30 tensor in the $\alpha = 1$ run

D DERIVATION

FIXED EXPERIMENT

900 spiral examples; data seed 7
30 hidden layers; width 128
He-normal weights; model seed 11
zero biases; before training

```
with torch.no_grad():
    _, H = models[1.0](X)

H30 = H[-1]                    # (900, 128)
H30.square().mean().sqrt()   # 1.096869
```

$$r_1 = \mathrm{RMS}\left(H_{30}^{(1)}\right) = \sqrt{\frac{1}{900\times128}\sum_{s=1}^{900}\sum_{j=1}^{128}\left(H_{30,sj}^{(1)}\right)^2} = 1.096869$$

This is a measured value from this seeded run, not a universal constant supplied by He initialization.

The reference RMS times α^{30} predicts the scaled runs

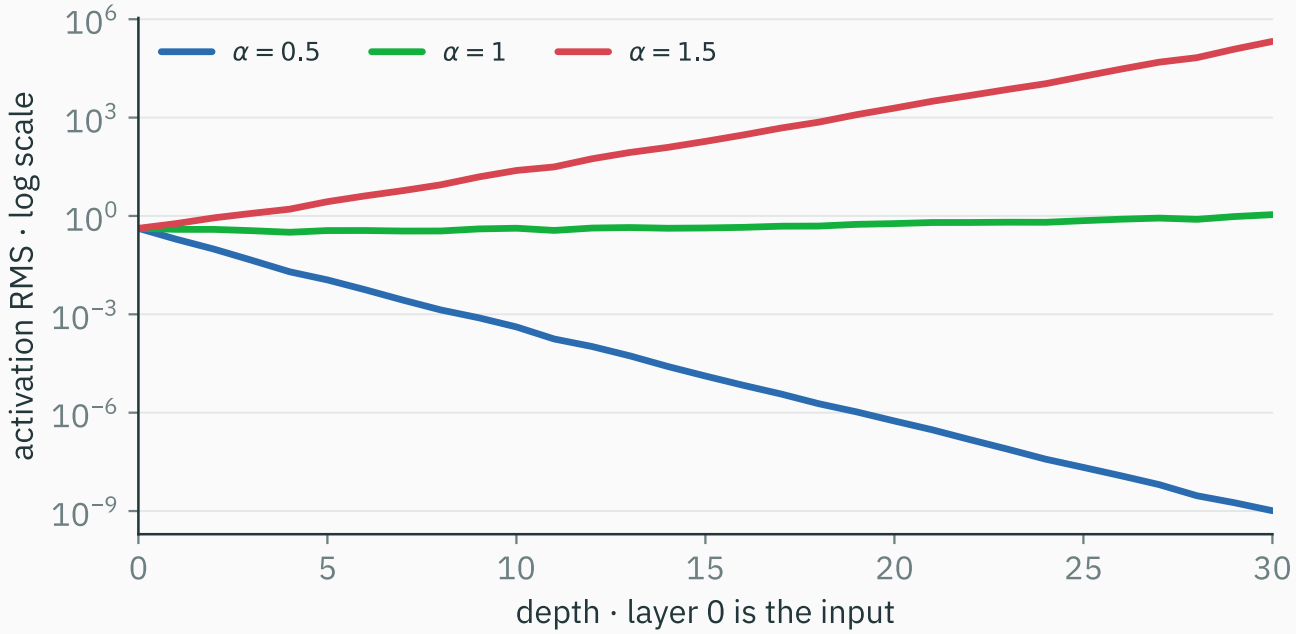
D DERIVATION

$$r_\alpha = \mathrm{RMS}\left(H_{30}^{(\alpha)}\right) = \alpha^{30} r_1, \quad r_1 = 1.096869$$

α	theoretical multiplier α^{30}	calibrated prediction $\alpha^{30} r_1$	direct forward-pass RMS
0.5	9.31323×10^{-10}	1.021539×10^{-9}	1.021539×10^{-9}
1.0	1	1.096869	1.096869
1.5	1.91751×10^{5}	2.103259×10^{5}	2.103258×10^{5}

The multiplier is derived. One measured reference fixes the absolute scale; the other forward passes check the prediction.

Activation RMS across all 30 hidden layers



$\alpha = 0.5$

The RMS loses roughly one factor of 0.5 per layer.

$\alpha = 1$

The RMS stays near order 1 through depth.

$\alpha = 1.5$

The RMS gains roughly one factor of 1.5 per layer.

The vertical axis is logarithmic; equal vertical gaps represent multiplicative differences.

Minimal executable notebook for this plot ↗

Activation-only notebook: return every post-ReLU tensor

I INTERACTIVE

```
def forward(self, x):
    H = []
    for layer in self.hidden:
        x = F.relu(layer(x))
        H.append(x)
    return self.output(x), H

with torch.no_grad():
    logits, H = model(X)
    trace = [rms(h) for h in H]
```

WHAT THE NOTEBOOK RECORDS

Open notebook ↗

30 post-ReLU tensors
each with shape 900×128
one trace for each α

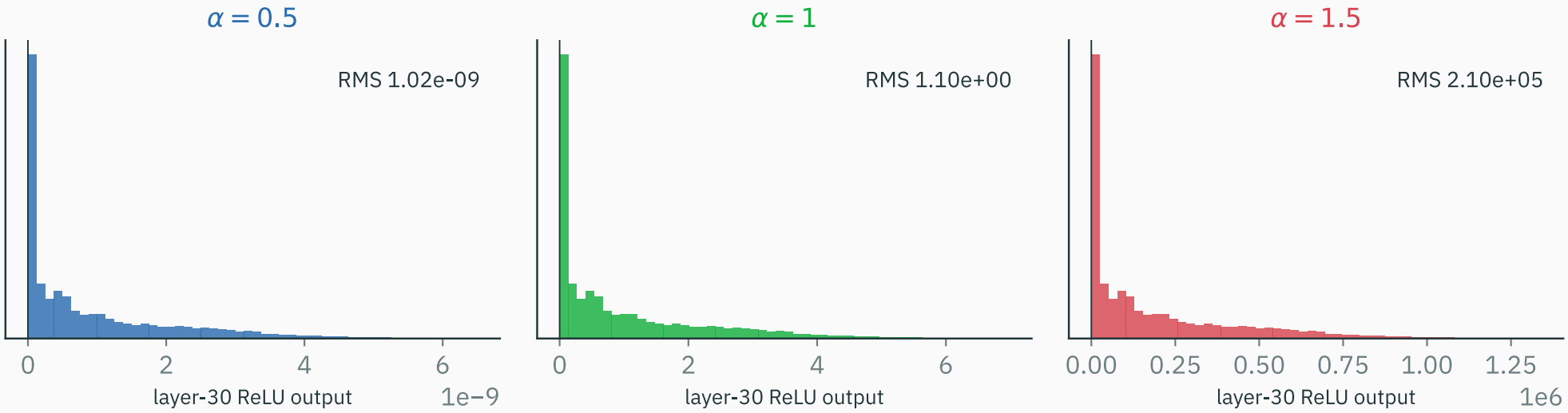
LAYER-30 RMS

$$1.02 \times 10^{-9}$$
$$1.0969$$
$$2.10 \times 10^{5}$$

This cell records activations under **torch.no_grad()**; the hooks tutorial records G_ℓ after backpropagation.

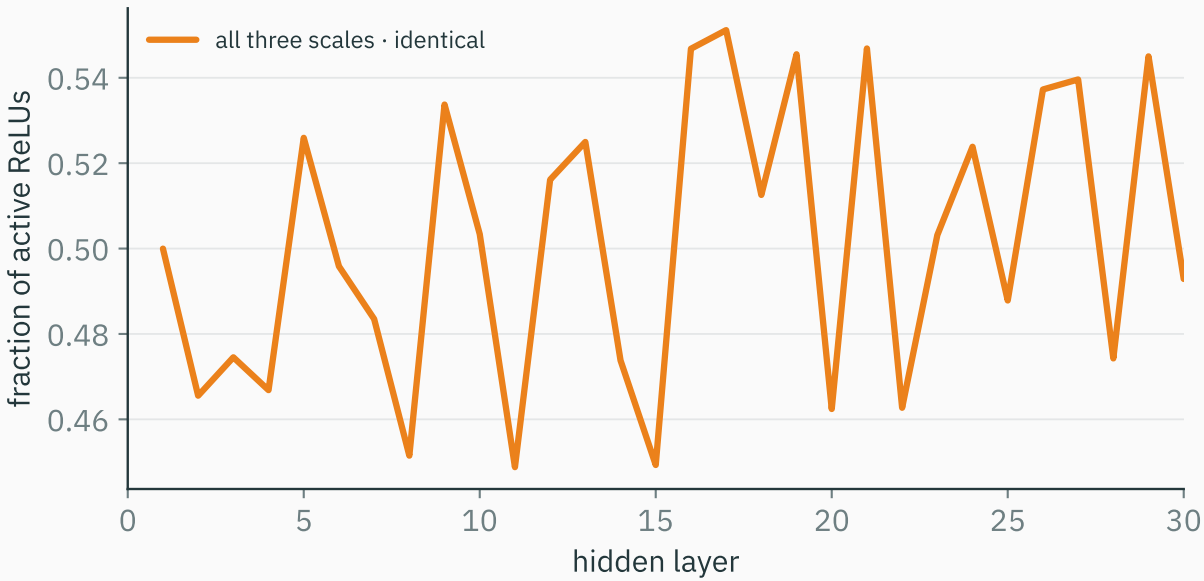
Layer-30 activation distributions

the same layer, three scales



Each panel uses its own horizontal scale; read the exponent on the axis. The RMS gap reflects the full distributions, not one outlier coordinate.

Active-ReLU fraction alone cannot diagnose signal scale



RULES OUT

At initialization, positive rescaling with zero biases preserves every gate. The $\alpha = 0.5$ run did not switch them all off.

STILL MISSES

Activation and gradient magnitudes can still vanish or explode.

Softmax converts three logits into class probabilities

D DERIVATION

Fixed spiral point: $x = (0.429, -0.410)$, true class $y = 0$. Start with the $\alpha = 1$ model.

1 · LOGITS

$$\boldsymbol{o} = (-1.313, -0.070, -1.047)$$

One score for each class.

$\rightarrow$

2 · SHIFT

$$\boldsymbol{o} - o_{\max} = (-1.243, 0, -0.977)$$

A common shift leaves softmax unchanged.

$\rightarrow$

3 · EXPONENTIATE

$$\exp(\boldsymbol{o} - o_{\max}) = (0.288, 1, 0.376)$$

Positive numbers; their sum is 1.664.

$\rightarrow$

4 · NORMALIZE

$$\boldsymbol{p} = (0.173, 0.601, 0.226)$$

$$p_k = \frac{\exp(o_k)}{\sum_j \exp(o_j)}$$

PREDICTION

Largest logit and probability: class 1.
The model predicts class 1.

CROSS-ENTROPY FOR THE TRUE CLASS

The truth is class 0, so
$\mathcal{L} = -\log p_0 = -\log(0.173) = 1.753.$

Positive scaling preserves class order, not confidence

D DERIVATION

Here all 31 weight matrices are scaled and biases are zero: $\boldsymbol{o}^{(\alpha)} = \alpha^{31}\boldsymbol{o}^{(1)}$ for $\alpha > 0$.

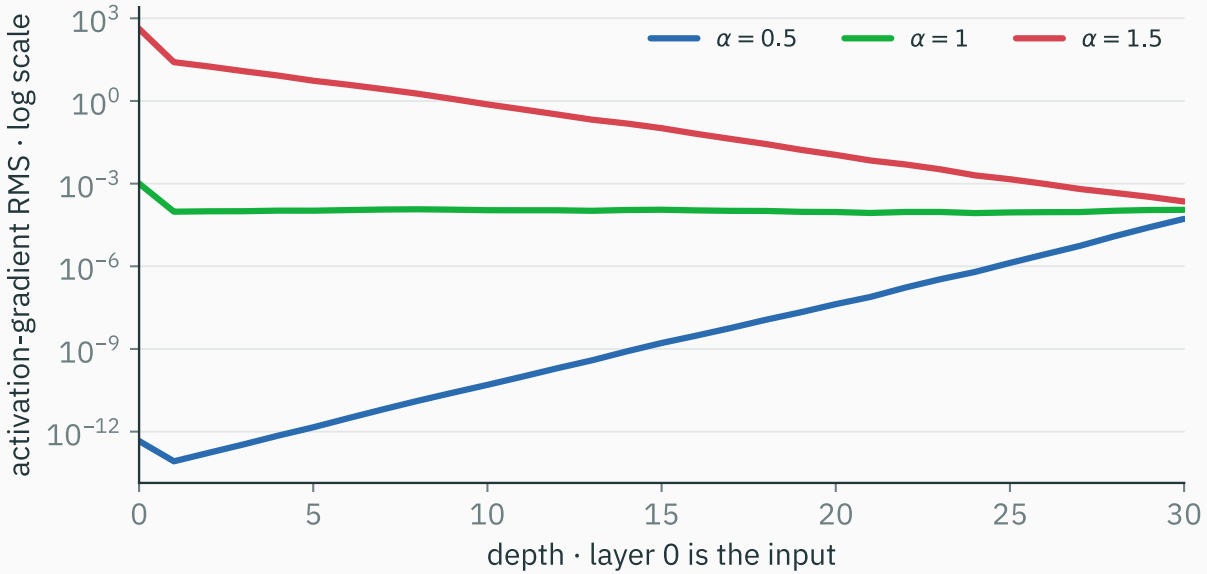
α	logits (o_0, o_1, o_2)	softmax (p_0, p_1, p_2)	pred.	one-point loss $-\log p_y$
0.5	$(-6.1 \times 10^{-10}, -3.2 \times 10^{-11}, -4.9 \times 10^{-10})$	$\approx (0.333, 0.333, 0.333)$	1	1.099
1.0	$(-1.313, -0.070, -1.047)$	$(0.173, 0.601, 0.226)$	1	1.753
1.5	$(-377633, -20019, -301145)$	$\approx (0, 1, 0)$	1	357614

Probabilities are rounded. PyTorch uses stable log-sum-exp, so the $\alpha = 1.5$ loss is finite even though p_0 displays as 0.

Positive scaling keeps $o_1 > o_2 > o_0$, so the prediction stays class 1. Softmax reads the gaps: larger gaps sharpen confidence. Because the true class is 0, sharpening magnifies this mistake.

Activation-gradient RMS should remain usable across depth

$G_\ell = \partial \frac{\mathcal{L}}{\partial} H_\ell$. Backpropagation travels from layer $30 \to 0$; read every curve from right to left.



$\alpha = 0.5$ · **VANISHES**

$\mathrm{RMS}(G_0) = 4.57 \times 10^{-13}$

$\alpha = 1$ · **USABLE**

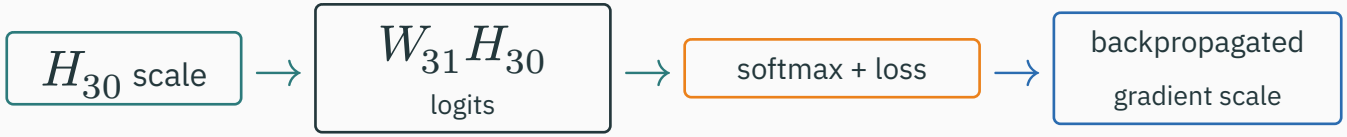
$\mathrm{RMS}(G_0) = 1.01 \times 10^{-3}$

$\alpha = 1.5$ · **EXPLODES**

$\mathrm{RMS}(G_0) = 4.15 \times 10^{2}$

Before training versus after 150 Adam updates

D DERIVATION



$\alpha = 0.5$ · COLLAPSED

BEFORE TRAINING layer-30 RMS:

1.02×10^{-9}

initial loss: $1.09861 \approx \log 3$

$\mathrm{RMS}(G_0)$: 4.57×10^{-13}

AFTER 150 UPDATES **accuracy**

33.3%

$\alpha = 1$ · REFERENCE

BEFORE TRAINING layer-30 RMS:

1.10

initial loss: 1.22422

$\mathrm{RMS}(G_0)$: 1.01×10^{-3}

AFTER 150 UPDATES **accuracy**

100%

$\alpha = 1.5$ · EXPLODED

BEFORE TRAINING layer-30 RMS:

2.10×10^{5}

initial loss: 1.89×10^{5}

$\mathrm{RMS}(G_0)$: 4.15×10^{2}

AFTER 150 UPDATES **accuracy**

99.4%

Earlier decision maps show update 50; these accuracies are at update 150. $G_0 = \partial \frac{\mathcal{L}}{\partial} X$ for each model's own loss; it is not a parameter gradient.

At $\alpha = 1.5$, finite logits already give saturated confidence

THE FIXED POINT FROM THE WORKED EXAMPLE

logits: $\boldsymbol{o} = (-377633, -20019, -301145)$

probabilities: $\boldsymbol{p} \approx (0, 1, 0)$

true class: 0; predicted class: 1

$$-\log p_0 = 357614$$

OVER ALL 900 TRAINING POINTS

all logits finite: **yes**

mean maximum probability: ≈ 1

mean cross-entropy: 1.89×10^5

The model starts extremely confident, often in the wrong class.

AFTER 150 ADAM UPDATES

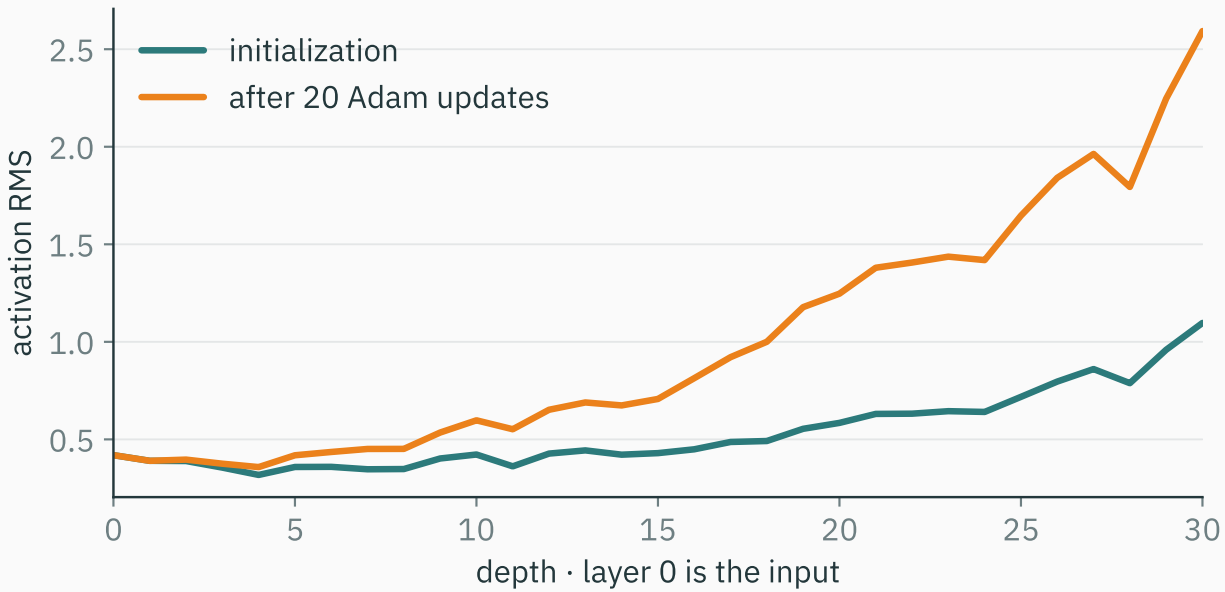
loss 0.0819; training accuracy 99.4%

Finite is not the same as well-scaled. Eventual recovery does not make the initialization healthy.

Normalization recomputes activation scale during training

He targets activation scale at initialization; training can move it

V VISUAL

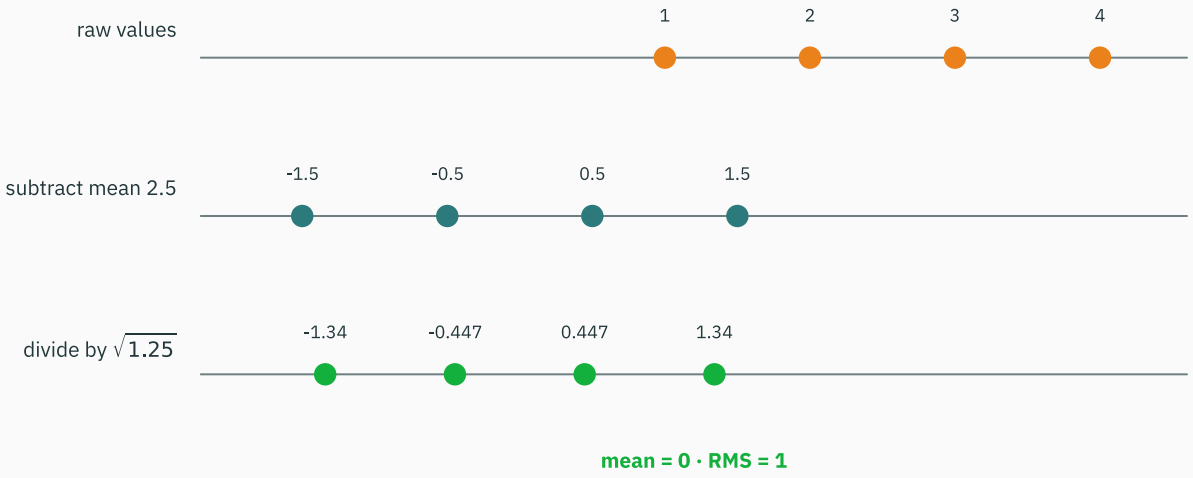


This is the same plain $\alpha = 1$ model at initialization and after 20 of its 150 Adam updates.

How can a layer use its current batch to reset centre and scale?

One hidden feature for four examples

$$Z[:, d] = (1, 2, 3, 4)^\top$$



WHAT THE SYMBOLS MEAN

d is one hidden feature. The four values come from examples 1–4.

THE GOAL

Shift the four values to mean 0, then divide by their standard deviation.

Step 1: subtract the feature mean

$$\mu_d = \frac{1+2+3+4}{4} = 2.5$$

BEFORE CENTRING

EXAMPLE 1	EXAMPLE 2	EXAMPLE 3	EXAMPLE 4
1	2	3	4

↓ **subtract** 2.5

EXAMPLE 1	EXAMPLE 2	EXAMPLE 3	EXAMPLE 4
-1.5	-0.5	0.5	1.5

The centred values have mean 0.

Step 2: divide by the feature standard deviation

$$\sigma_d^2 = \frac{(-1.5)^2+(-0.5)^2+0.5^2+1.5^2}{4} = 1.25, \quad \sigma_d = \sqrt{1.25}$$

-1.5	-0.5	0.5	1.5

↓ divide by $\sqrt{1.25}$

-1.342	-0.447	0.447	1.342

Ignoring ε for this hand calculation: $\mathrm{mean}\big(\hat{Z}[:,d]\big) = 0, \quad \mathrm{RMS}\big(\hat{Z}[:,d]\big) = 1$. After centring, standard deviation and RMS agree.

Code uses $\varepsilon > 0$ in the denominator.

Apply learned scale and shift after standardization

Start from the standardized values. For one feature, choose $\gamma_d = 2$ and $\beta_d = 1$.

-1.342	-0.447	0.447	1.342

↓ $y_{nd} = 2\hat{z}_{nd} + 1$

-1.683	0.106	1.894	3.683

γ_d: **LEARNED FEATURE SCALE**

restores or changes the standardized scale

β_d: **LEARNED FEATURE OFFSET**

restores or changes the standardized centre

Normalization chooses which entries share statistics

For a batch matrix $Z \in \mathbb{R}^{B \times D}$, let S_{nd} contain the entries that share statistics with position (n, d).

$$\mu_{nd} = \frac{1}{|S_{nd}|} \sum_{(r,k) \in S_{nd}} Z_{rk}, \quad \sigma^2_{nd} = \frac{1}{|S_{nd}|} \sum_{(r,k) \in S_{nd}} (Z_{rk} - \mu_{nd})^2.$$

$$\hat{Z}_{nd} = \frac{Z_{nd} - \mu_{nd}}{\sqrt{\sigma^2_{nd} + \varepsilon}}, \quad Y_{nd} = \gamma_d \hat{Z}_{nd} + \beta_d.$$

BATCHNORM CHOICE

$$S_{nd} = \{(r, d) : r = 1, ..., B\}$$

LAYERNORM CHOICE

$$S_{nd} = \{(n, k) : k = 1, ..., D\}$$

The set changes; the learned affine parameters remain feature-indexed: γ_d, β_d.

Rows are examples; columns are hidden features

examples $n = 1, \ldots, B$

hidden features $d = 1, \ldots, D$

1	2	6
2	4	5
3	8	4
4	10	3

$Z_\ell \in \mathbb{R}^{B \times D}$: preactivations before ReLU, here $B = 4$ and $D = 3$.

For $(Z_\ell)_{1,1} = 1$, should $S_{1,1}$ be its column or its row?

For an MLP batch, BatchNorm uses one feature across examples

1	2	6
2	4	5
3	8	4
4	10	3

The first feature column is the same four-value worked set $[1, 2, 3, 4]$:

$\hat{Z}[:, 1] = (-1.342, -0.447, 0.447, 1.342)^\top$.

BatchNorm computes μ_d, σ_d over examples; each feature has learned γ_d, β_d.

LayerNorm over D features uses one example at a time

1	2	6
2	4	5
3	8	4
4	10	3

For the first row $n = 1$, whose features are $[1, 2, 6]$:

$$\mu_{n=1}^{\mathrm{LN}} = 3, \quad \left(\sigma_{n=1}^{\mathrm{LN}}\right)^2 = \frac{14}{3}, \quad \sigma_{n=1}^{\mathrm{LN}} = \sqrt{\frac{14}{3}},$$

$$\hat{z}_{1\cdot} = [-0.926, -0.463, 1.389].$$

The statistics are per example; the learned affine parameters are normally indexed by feature.

BatchNorm and LayerNorm reduce different axes

BATCHNORM · ONE FEATURE COLUMN

1	2	6
2	4	5
3	8	4
4	10	3

statistics over examples depend on the other rows during training

LAYERNORM · ONE EXAMPLE ROW

1	2	6
2	4	5
3	8	4
4	10	3

statistics over features does not use the other examples

Same matrix, different reduction axis.

The normalized value of 1 depends on its training batch

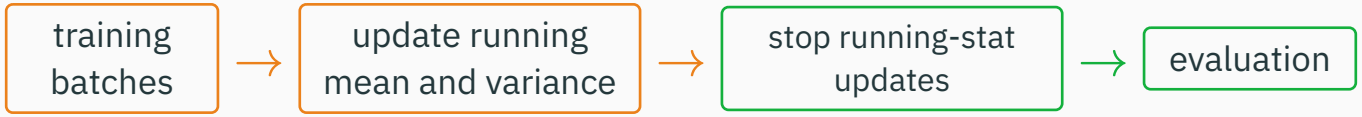
Keep the raw value $z = 1$ fixed and change only its companions.

BATCH A

BATCH B

The raw value is unchanged. Its normalized value changes because the training batch changed.

Evaluation mode uses stored training statistics



`MODEL.TRAIN()`

use batch statistics and update running estimates

`MODEL.EVAL()`

use the stored running estimates

model.eval() does not freeze parameters or disable autograd. LayerNorm uses the same per-example computation in both modes.

In this experiment: Linear → BatchNorm → ReLU

$Z_\ell = H_{\ell-1} W_\ell^\top + \mathbf{1} b_\ell^\top$. Each H and Z is a batch matrix; BatchNorm computes column statistics during training.

```
z = linear(h)
h = torch.relu(batch_norm(z))
```

After BatchNorm and ReLU, activation RMS is about 0.707

STANDARDIZED PREACTIVATIONS

-1.342	-0.447	0.447	1.342

↓ ReLU

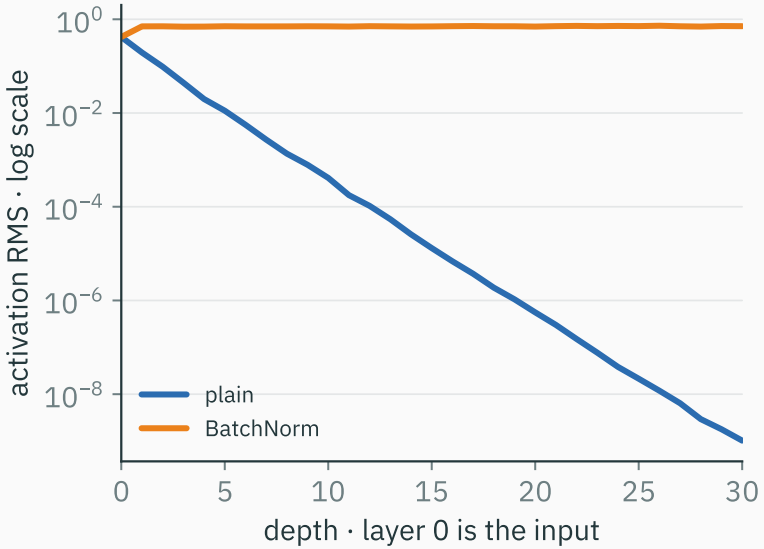
0	0	0.447	1.342

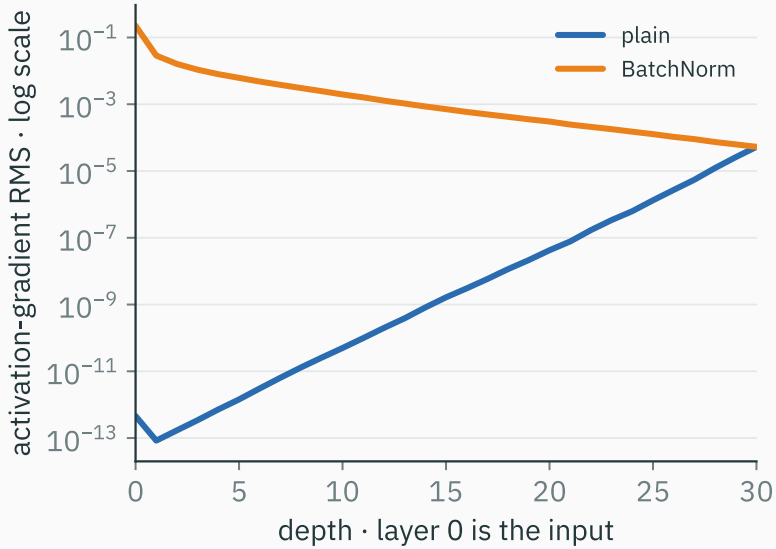
$$\mathrm{RMS}(H) = \sqrt{\frac{0+0+0.2+1.8}{4}} = \sqrt{\frac{1}{2}} = 0.707$$

Prediction at initialization: with $\gamma = 1$, $\beta = 0$, and roughly symmetric normalized preactivations, every hidden layer should report RMS near 0.707.

In this 30-hidden-layer run, BatchNorm keeps activation RMS near 0.7

Activation-gradient RMS need not equal 1; compare collapse with a usable depthwise signal. Read the gradient panel from layer $30 \to 0$.





PLAIN · $\alpha = 0.5$

layer-30 RMS $= 1.02 \times 10^{-9}$

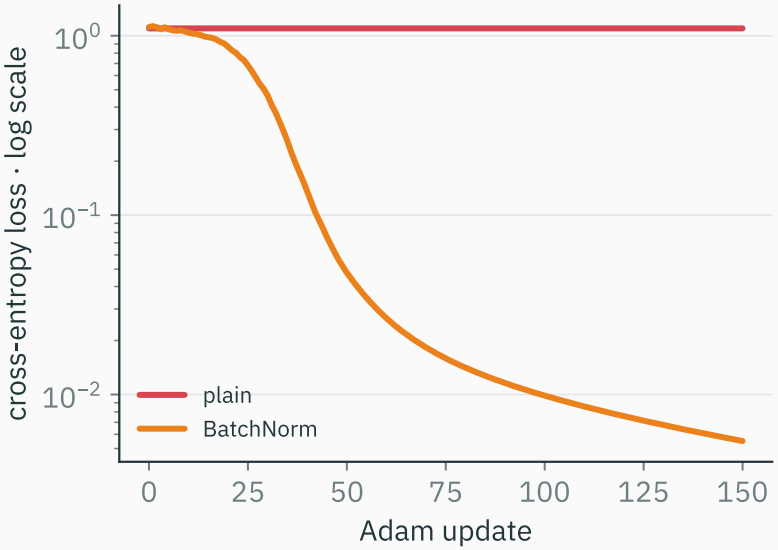
$\mathrm{RMS}(G_0) = 4.57 \times 10^{-13}$

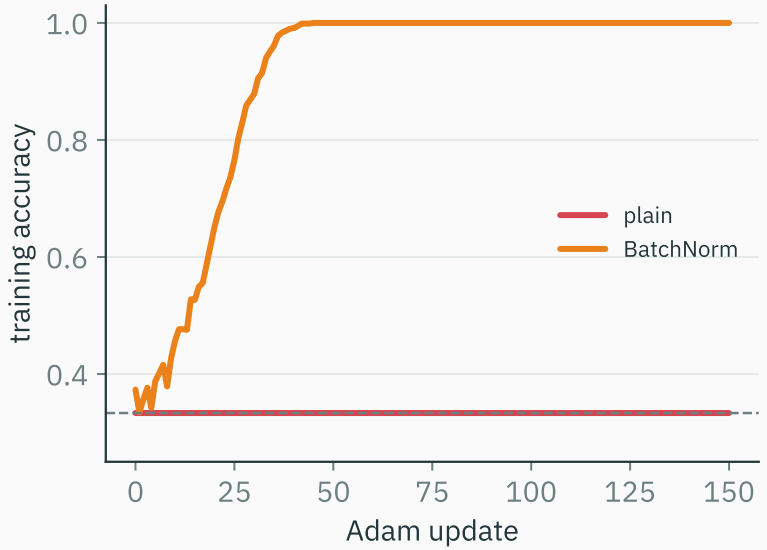
BATCHNORM · $\alpha = 0.5$

layer-30 RMS ≈ 0.713

$\mathrm{RMS}(G_0) \approx 0.225$

With $\alpha = 0.5$, BatchNorm reaches 100% training accuracy



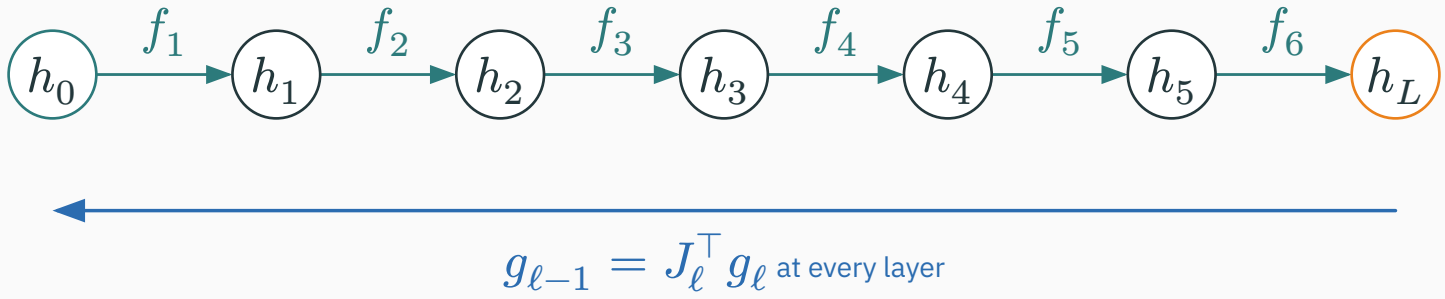


All 900 examples form one batch, so this run removes mini-batch-composition noise while retaining BatchNorm's normalization effect.

Residual blocks add an identity term to the local Jacobian

Why add an identity path?

V VISUAL



Scalar schematic; the same product-of-Jacobians issue applies to vector activations.

For a parameter in layer 1, the gradient still crosses the Jacobian of every later layer.

LOCAL QUESTION

Are the activation and gradient magnitudes usable at each layer?

→

ROUTE QUESTION

Must useful information pass through every learned transformation?

Can a deep block simply copy its input?

Take the scalar input $h_\ell = 3$ and target $y = 3$.

PLAIN SCALAR LAYER

$$\hat{y} = w h_\ell$$

$\hat{y} = 3$ only when $w = 1$.

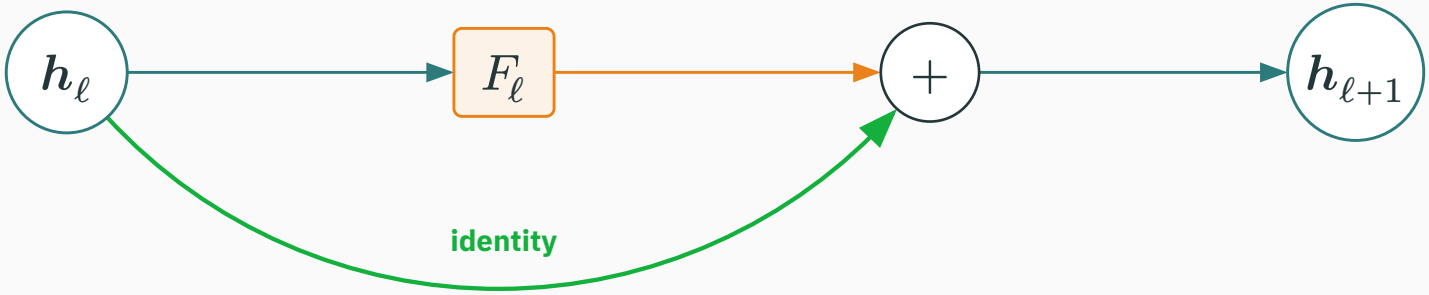
IDENTITY PLUS CORRECTION

$$\hat{y} = h_\ell + w h_\ell$$

At $w = 0$, $\hat{y} = 3$ already.

The residual parameterization represents identity before the learned branch does any work.

A residual block adds a correction with the same shape



$$h_{\ell+1} = h_\ell + F_{\ell(h_\ell)}$$

SHAPE REQUIREMENT

$$h_\ell, F_{\ell(h_\ell)} \in \mathbb{R}^D$$

ONE NUMERIC ADDITION

$$\begin{pmatrix} 2 \\ -1 \end{pmatrix} + \begin{pmatrix} 0.3 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 2.3 \\ -0.8 \end{pmatrix}$$

In the scalar example, the local gradient factor is $1 + F'_{\ell(h_\ell)}$

Let $h_{\ell+1} = h_\ell + F_{\ell(h_\ell)}$, choose the explicit branch $F_{\ell(h)} = -0.1h$, and let the arriving gradient be $g_{\ell+1} = \partial \frac{\mathcal{L}}{\partial} h_{\ell+1} = 1$.

$$g_\ell = \frac{\partial \mathcal{L}}{\partial h_\ell} = g_{\ell+1}\left(1 + F'_{\ell(h_\ell)}\right)$$

$$g_\ell = 1(1 - 0.1) = 0.9$$

This scalar branch is chosen only to expose the derivative. It is not the ReLU branch used in the spiral experiment.

The vector derivative contains an identity term

For $\boldsymbol{h}_\ell \in \mathbb{R}^D$, define

$$J_{F_\ell} = \frac{\partial F_{\ell(\boldsymbol{h}_\ell)}}{\partial \boldsymbol{h}_\ell} \in \mathbb{R}^{D\times D}, \quad \boldsymbol{g}_\ell = \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_\ell} \in \mathbb{R}^D.$$

$$\frac{\partial \boldsymbol{h}_{\ell+1}}{\partial \boldsymbol{h}_\ell} = I + J_{F_\ell}$$

$$\boldsymbol{g}_\ell = \left(I + J_{F_\ell}\right)^\top \boldsymbol{g}_{\ell+1} = \underbrace{\boldsymbol{g}_{\ell+1}}_{\text{identity term}} + \underbrace{J_{F_\ell}^\top \boldsymbol{g}_{\ell+1}}_{\text{branch term}}$$

Addition requires matching shapes; otherwise the skip needs a projection.

Over five blocks: 10^{-5} versus 0.59049

Continue the scalar example with $F_\ell' = -0.1$ at every block and the same unit arriving gradient.

BRANCH-ONLY PARAMETERIZATION

derivative magnitude per block: 0.1

$$\left|(-0.1)^5\right| = 10^{-5}$$

IDENTITY + BRANCH

derivative per block: $1 - 0.1 = 0.9$

$$(0.9)^5 = 0.59049$$

This is an abstract comparison of two parameterizations, not a measured claim about all residual networks. A large branch can amplify the signal, and $I + J_F$ can still cancel along some direction.

In this experiment, $F_{\ell(\boldsymbol{h})} = \mathrm{ReLU}(V_\ell \boldsymbol{h})$

STEM CHANGES WIDTH

$$\boldsymbol{x} \in \mathbb{R}^2 \to \boldsymbol{h}_1 \in \mathbb{R}^{128}$$
$$\boldsymbol{h}_1 = \mathrm{ReLU}(W_{\mathrm{stem}} \boldsymbol{x})$$

$\rightarrow$

29 MATCHING-WIDTH BLOCKS

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + \mathrm{ReLU}(V_\ell \boldsymbol{h}_\ell)$$
$$\ell = 1, \ldots, 29$$

Away from a ReLU kink, $J_{F_\ell} = D_\ell V_\ell$ with $D_\ell = \mathrm{diag}(\mathbf{1}[V_\ell \boldsymbol{h}_\ell > 0])$.

This is an isolation experiment, not a standard modern ResNet block. There is no activation after the addition. The nonnegative branch corrections can make activations large; the identity addition isolates the alternate route.

Compare three versions of the same 30-hidden-layer spiral model

PLAIN

30 ×
$\text{Linear} \to \text{ReLU}$

BATCHNORM

30 ×
$\text{Linear} \to \text{BatchNorm} \to \text{ReLU}$

RESIDUAL TOY

one $2 \to 128$ stem
then 29 × $\boldsymbol{h} + \text{ReLU}(V_\ell \boldsymbol{h})$

Fixed: spiral data, depth, width, loss, Adam at 3×10^{-4}, seed, and corresponding $\alpha = 0.5$ base weight directions.

PLAIN PREDICTION

scale should collapse through depth

BATCHNORM PREDICTION

current activation scale should be controlled

RESIDUAL-TOY PREDICTION

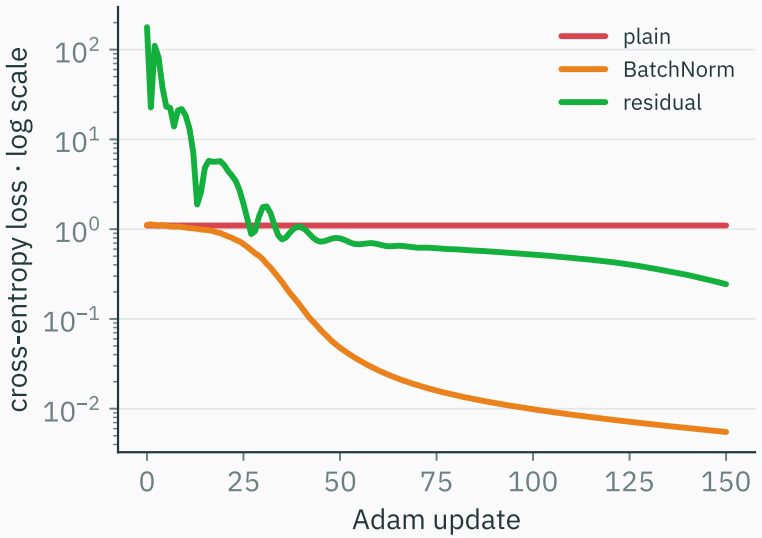
an identity gradient term remains, but positive corrections may enlarge activations

Measured at initialization for the three model variants

route	layer-30 activation RMS	$\mathrm{RMS}(G_0)$ $G_0 = \partial \frac{\mathcal{L}}{\partial} X$	initial loss
plain	1.02×10^{-9}	4.57×10^{-13}	1.09861
BatchNorm	0.713	0.225	1.11666
residual toy	698.57	0.329	177.84

BatchNorm controls activation scale. The residual measurements are consistent with a usable backward route, while the $I + J_F$ derivation identifies the mechanism; this minimal branch also starts with very large activations.

Training results for the controlled comparison



PLAIN

33.3%

BATCHNORM

100%

RESIDUAL TOY

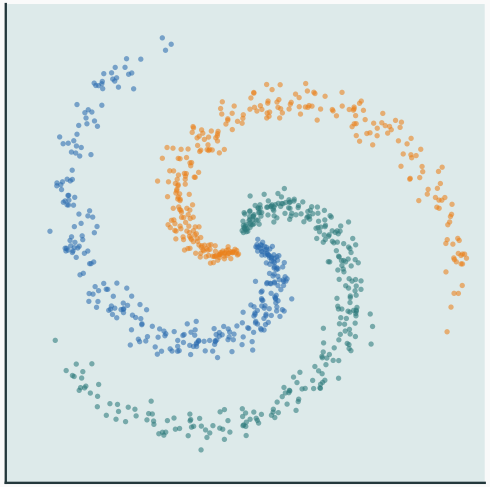
98.6%

Final training accuracy after 150 updates. This controlled example compares mechanisms, not architectures in general.

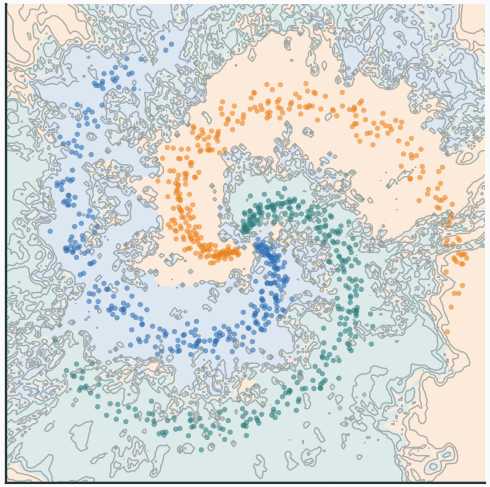
Decision regions after 150 updates

same small $\alpha = 0.5$ weights · change only the route

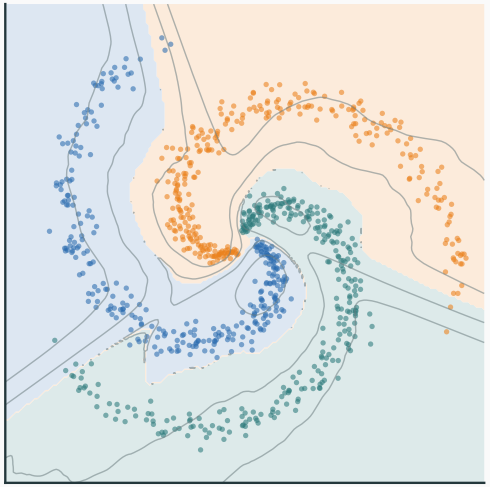
plain · flat



BatchNorm · fit



identity skips · fit



Data, optimizer, learning rate, seed, base weight directions, and training budget are fixed. The computation mechanism through the hidden layers changes.

This experiment demonstrates optimization and trainability on the training set; the jagged BatchNorm boundary is not evidence of better generalization.

Diagnose the failing layer before changing Adam

At $\alpha = 0.5$, scale collapses while half the ReLUs remain active

D DERIVATION

layer:	1	10	20	30
act RMS:	1.95e-1	4.13e-4	5.57e-7	1.02e-9
active:	0.50	0.50	0.46	0.49
finite:	yes	yes	yes	yes

SYMPTOM

Representation scale collapses with depth.

EVIDENCE AGAINST WIDESPREAD GATE CLOSURE

The aggregate active fraction stays near 0.5; individual neurons could still be dead.

FIRST SUSPECT

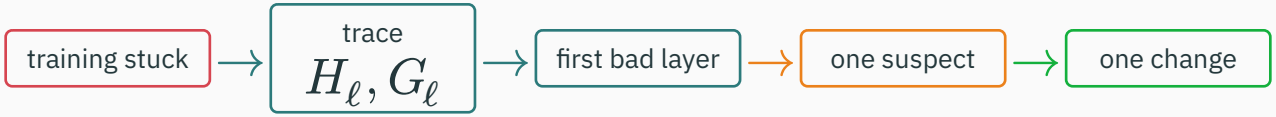
Per-layer weight gain is too small.

Test the scale hypothesis without changing weight directions

α	layer-30 activation RMS	$\mathrm{RMS}(G_0)$ $G_0 = \partial \frac{\mathcal{L}}{\partial} X$	active fraction
0.5	1.02×10^{-9}	4.57×10^{-13}	0.493
1.0	1.10	1.01×10^{-3}	0.493

At initialization, zero biases and positive rescaling preserve the gates; activation and gradient scales change. This supports the gain diagnosis.

Measure → locate → change one factor → measure again



SCALE

RMS shrinks or grows systematically.

GATES

Near 0: gates close. Near 1: preactivations shifted positive and ReLU is nearly linear.

NUMERICS

Find the first non-finite layer.

Rerun the same measurements; do not judge the repair from loss alone.

Exit ticket: diagnose a different trace

A new 30-hidden-layer ReLU MLP reports

```
layer:        1       10       20       30
act RMS:   0.80     0.72     0.65     0.60
active:    0.49     0.31     0.08     0.01
finite:     yes      yes      yes      yes
```

Write three lines:

1. the **symptom** in plain language;
2. the **first suspect**;
3. the **smallest informative measurement or one-factor test**.

Exit-ticket answer: the ReLU gates progressively close

SYMPTOM

Activation RMS falls modestly, but the active fraction collapses from 0.49 to 0.01.

FIRST SUSPECT

The preactivation distribution is drifting to the negative side, closing almost every ReLU gate.

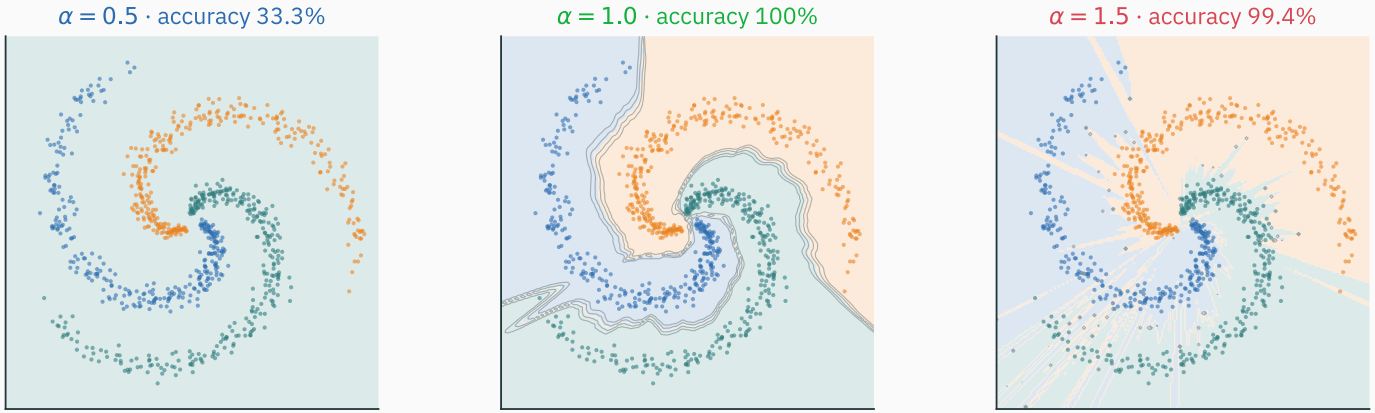
SMALLEST TEST

Plot the preactivation mean and histogram by layer; locate where the negative shift begins.

This trace is a gate problem, not the $\alpha = 0.5$ scale-collapse pattern: the active fraction is the distinguishing evidence.

The same measurements explain all three α runs

after 150 Adam updates



$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 1.5$
forward and backward scales collapse	usable scales across depth	very large but finite scales

For a new run: trace activations and gradients, locate the first break, change one cause, and repeat the same measurements.

Primary sources and reproducible evidence

Opening intuition

DeepLearning.AI: Vanishing/Exploding Gradients
DeepLearning.AI: Weight Initialization in a Deep Network ↗

Initialization

Glorot & Bengio (2010), Xavier initialization ↗
He et al. (2015), rectifier-aware initialization ↗

Shortcuts

He et al. (2016), Deep Residual Learning ↗

Normalization

Ioffe & Szegedy (2015), Batch Normalization ↗
Ba, Kiros & Hinton (2016), Layer Normalization ↗

This lecture's evidence

Minimal scale-through-depth notebook ↗
Executed PyTorch diagnostic notebook ↗
PyTorch hooks tutorial ↗
Hidden-unit symmetry lab ↗
Evidence tables, scripts, and figures ↗

All spiral figures were generated from the fixed-seed experiment used in the notebook.

Optional · Unequal input units can dominate the first layer

One neuron receives age 40 and annual income **2,000,000**. Suppose both weights are 0.2.

AGE CONTRIBUTION

$$0.2(40) = 8$$

vs

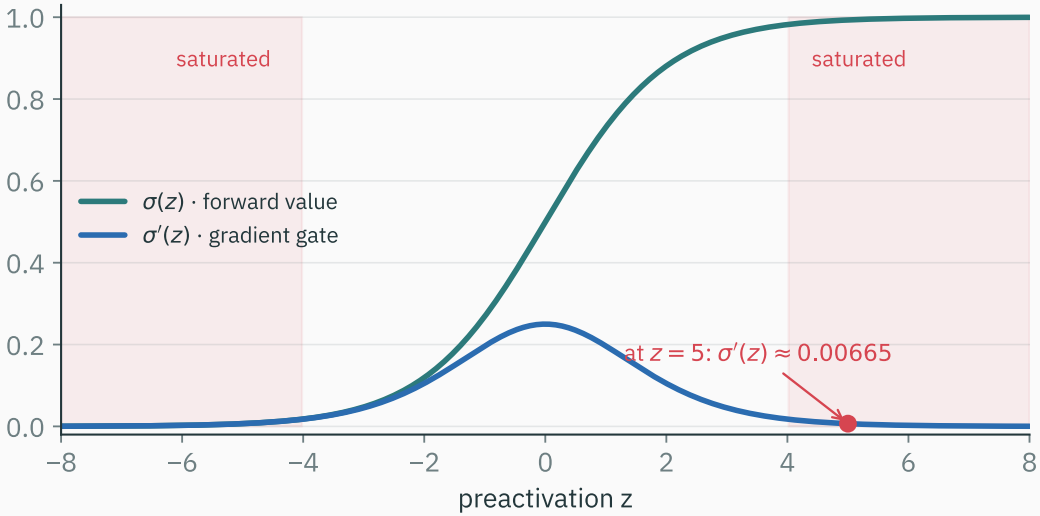
INCOME CONTRIBUTION

$$0.2 \times 2{,}000{,}000 = 400{,}000$$

After standardization, values 0.5 and -0.5 contribute 0.1 and -0.1.

Initialization formulas assume incoming coordinates already have comparable, roughly order-one scale.

Optional · Ten saturated sigmoid gates



At $z = 5$,

$$\sigma'(5) = \sigma(5)(1 - \sigma(5)) \approx 0.00665.$$

With ten gates all at $z = 5$ and intervening scalar weight factors fixed at 1, the gate-only multiplier is

$$1 \to 6.65 \times 10^{-3} \to 4.42 \times 10^{-5} \to \dots$$

$$(0.00665)^{10} \approx 1.7 \times 10^{-22}.$$

Sigmoid remains appropriate for a binary output; repeated saturated hidden gates are the issue.

Optional · The PyTorch initialization line

```
linears = [m for m in model.modules()
           if isinstance(m, nn.Linear)]

for m in linears[:-1]:          # hidden layers
    nn.init.kaiming_normal_(
        m.weight,
        mode="fan_in",
        nonlinearity="relu",
    )
    if m.bias is not None:
        nn.init.zeros_(m.bias)

with torch.no_grad():            # final classifier
    nn.init.xavier_uniform_(linears[-1].weight)
    if linears[-1].bias is not None:
        nn.init.zeros_(linears[-1].bias)
```

Use ReLU-aware Kaiming for hidden layers; initialize the final classifier separately.

Optional diagnosis · RMS is usable, but gates close

```
layer:        1       10       20       30
act RMS:     0.8      0.7      0.6      0.5
active:     0.49     0.31     0.08     0.01
```

Symptom

ReLU gates progressively close.

First suspect

Preactivations drifted negative.

Smallest test

Plot preactivation histograms and means.

Optional diagnosis · First non-finite activation at layer 17

```
layer 15: finite=True   max|h|=8.2e+14
layer 16: finite=True   max|h|=3.7e+28
layer 17: finite=False  max|h|=inf
```

Symptom

Numerical overflow begins inside the model.

First suspect

Excess forward gain before layer 17.

Smallest test

Inspect layers 15–17 before changing Adam.